Hands-on session: programming

University of Antwerp

Prof. Toon Calders toon.calders@uantwerpen.be

Goal of this lecture

Get a *feeling* of what is programming

- Focus is on *understanding*
- Writing *small snippets of code*

This session is hands-on

1. Introduction of a new concept
2. One exercise together
3. Small exercise(s) for you

Goal of this lecture

Learn the basic principles of coding

- Sequence of instructions
- Loops and repetitions
- Conditional execution
- Variables

We will use the following examples

Part 1 : Basics (maze)

Part 2 : Variables (loan payments)

Part 3 : Coding decision trees (GDPR)

We Will Use Google Colab

1. Create a Google account: https://accounts.google.com/
   Skip this step if you already have one
2. Sign in to your Google account (same link)

3. Follow this link: https://colab.research.google.com/drive/1WrrJ-9o27sMXnGXfcZ1NBW4OuwSFM5dR?usp=sharing
4. Save a copy of the notebook in your google drive folder

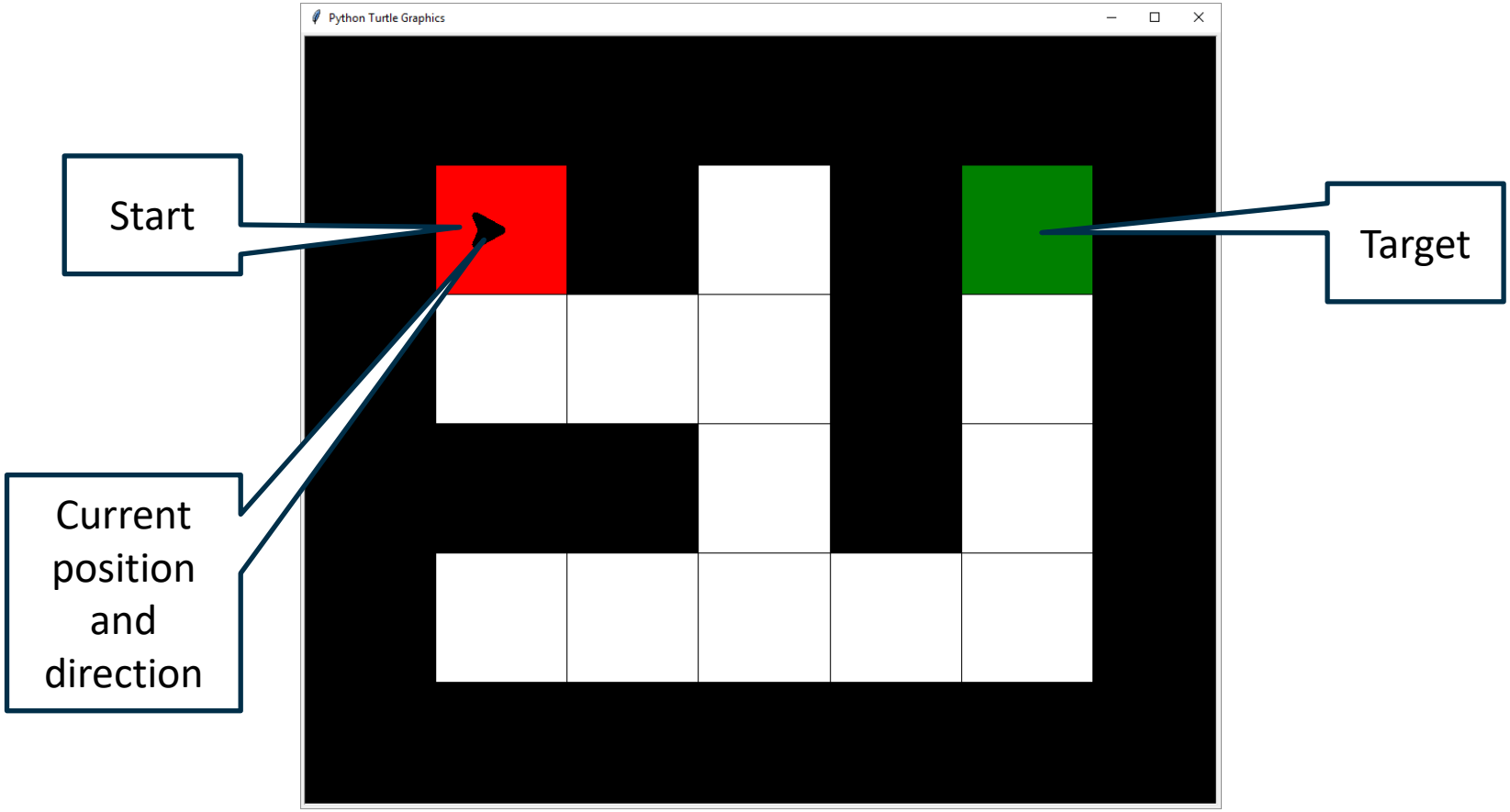
5. Start by executing the first two cells

Part 1 – Basic Constructions

University of Antwerp

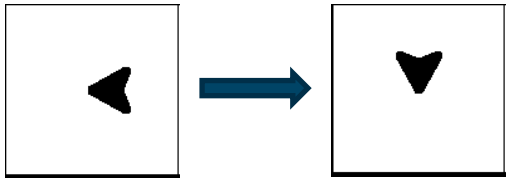
In the first part we will write …

… a Python program for guiding a robot out of a maze

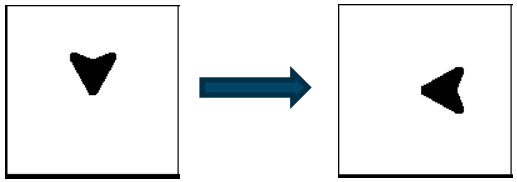


The instructions:

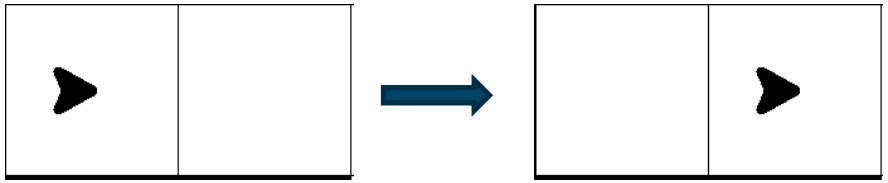
turnLeft()



turnRight()

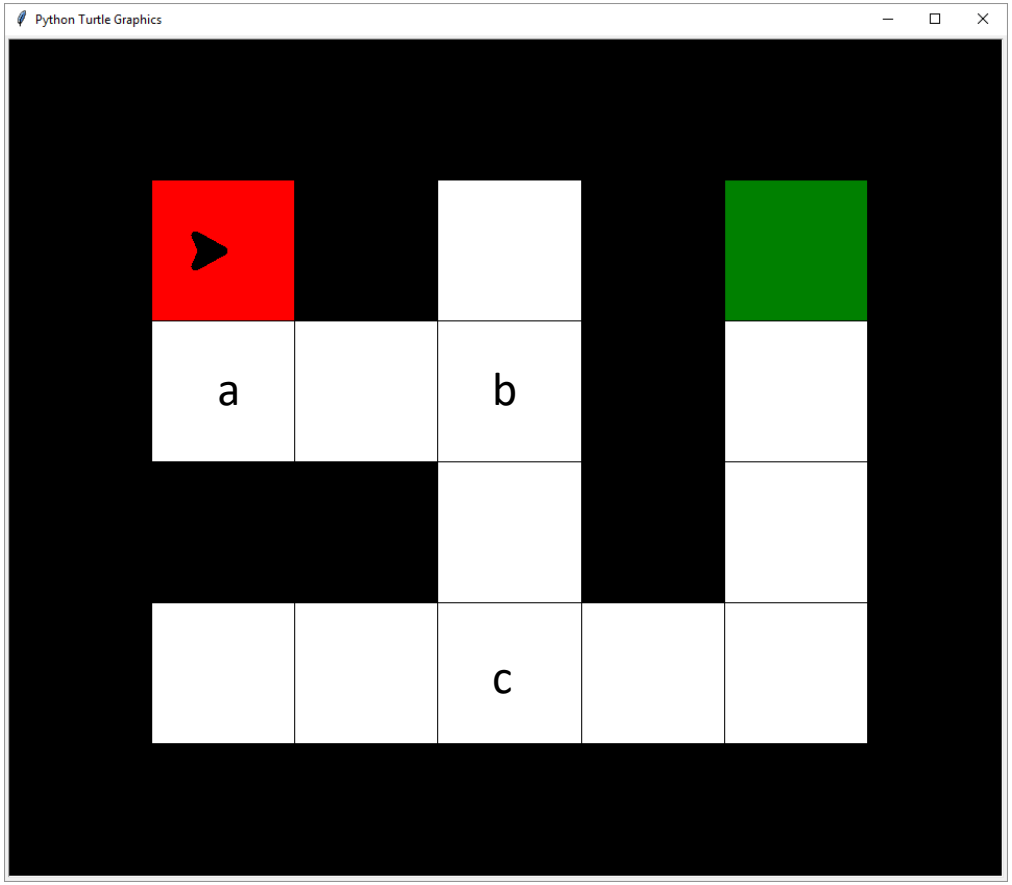


goForward()



Quick question

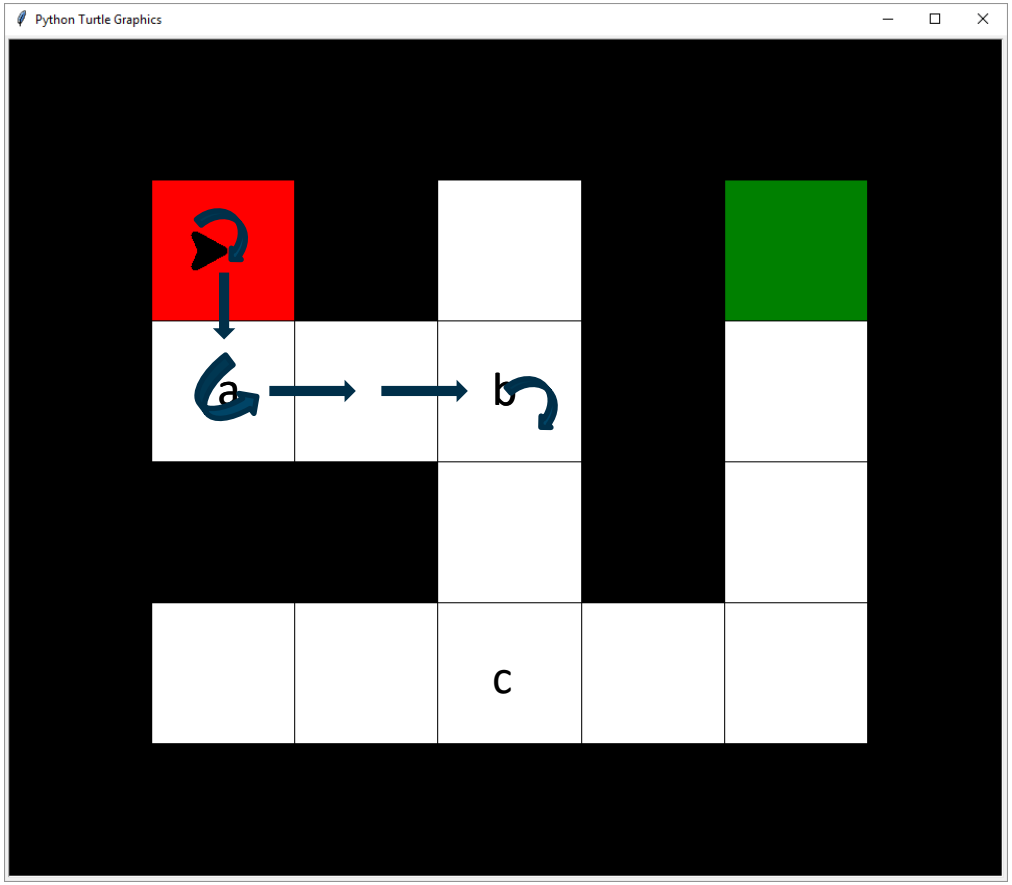
Consider the following starting position; what is the position of the robot after the commands on the right ?




```
turnRight()
goForward()
turnLeft()
goForward()
goForward()
turnRight()
```

Quick question

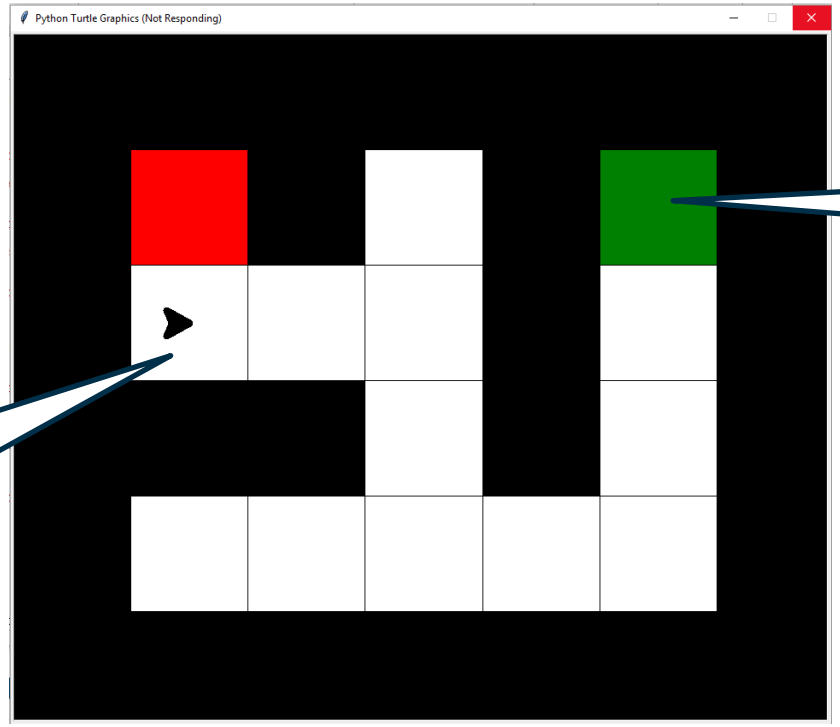
Consider the following starting position; what is the position of the robot after the commands on the right ?




```
turnRight()
goForward()
turnLeft()
goForward()
goForward()
turnRight()
```

Assignment #1 (q1.py)

Finish the program !



But I'd like to be here!

The program brings me till here

Solution Assignment #1

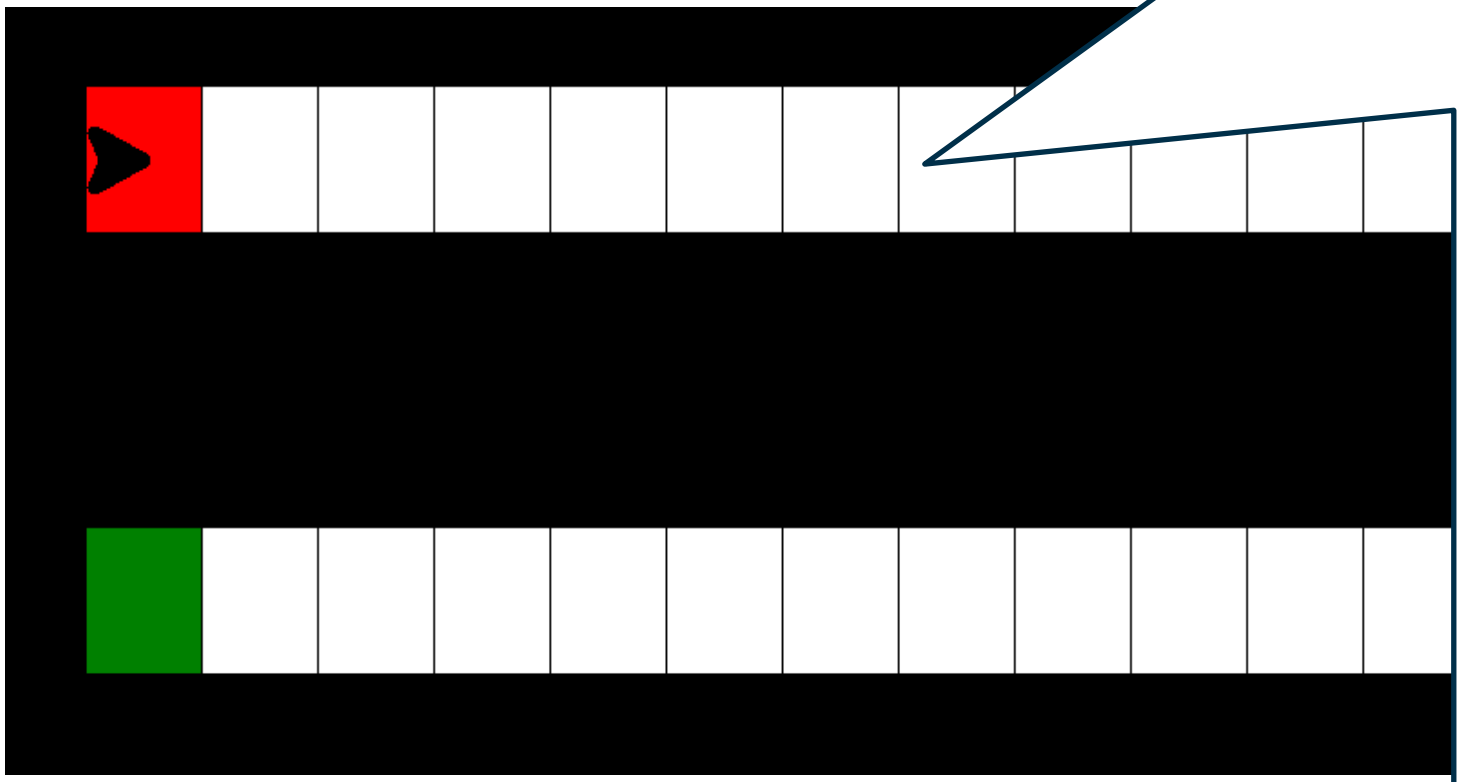
The next sequence of instructions gets you to the end:

```
from maze import *

# Problem 1:
loadMaze("maze1.txt")
turnRight()
goForward()
turnLeft()
goForward()
goForward()
turnRight()
goForward()
goForward()
turnLeft()
goForward()
goForward()
turnLeft()
goForward()
goForward()
goForward()
```

We are repeating ourselves …

Load the second question and execute (q2a.py)

```
# Question 2a: Solve maze with while
loadMaze("maze2a.txt")
```

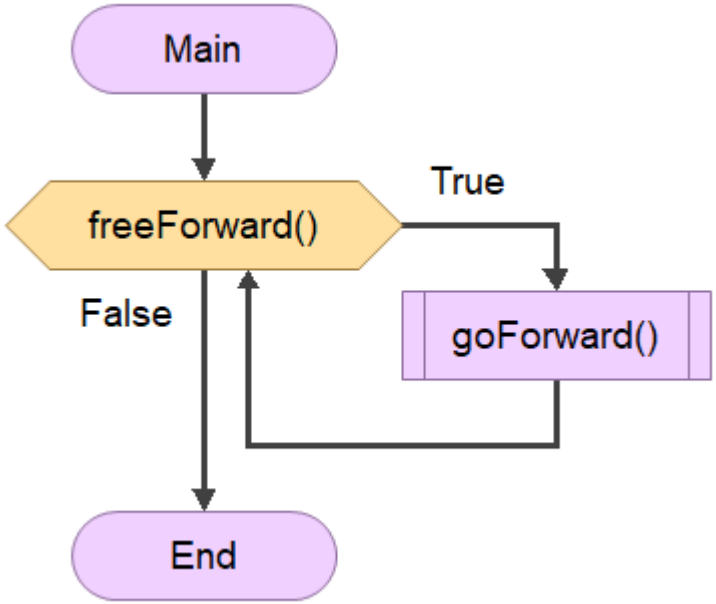


```
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
turnRight()
goForward()
goForward()
goForward()
turnRight()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
```

Repetition loop

"As long as you can go straight ahead, proceed and go forward"

```
while freeForward():
    goForward()
```



You can repeat one instruction (goForward()), but also a set of instructions

Use indent (tab!) for the lines to be repeated

New *functions*

function	True if …
freeForward()	… the place ahead is free
freeLeft()	… the place to your left is free
freeRight()	… the place to your right is free
	(always respective to your orientation)
foundExit()	True only if you are on the target tile
not *function()*	True if *function()* is False ; False otherwise

Example: Repetition Loop

```
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
goForward()
turnRight()
...
```

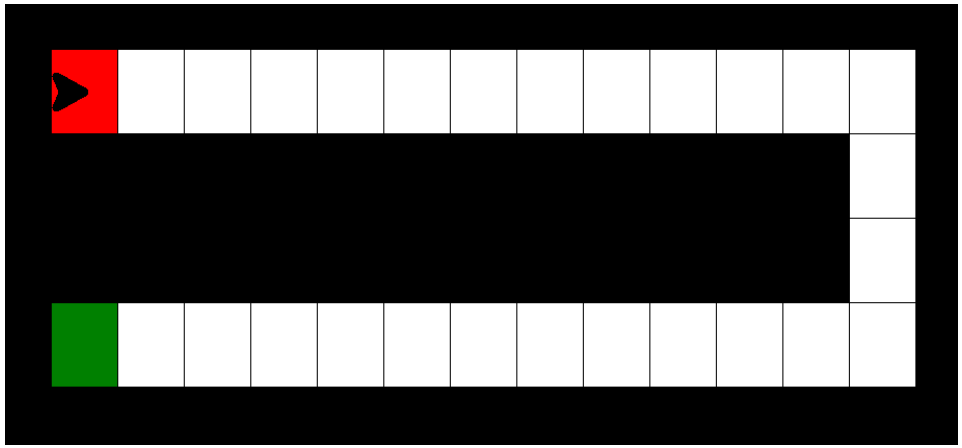
The part to be repeated is indented; use <TAB>

Lower case

Do not forget ":"

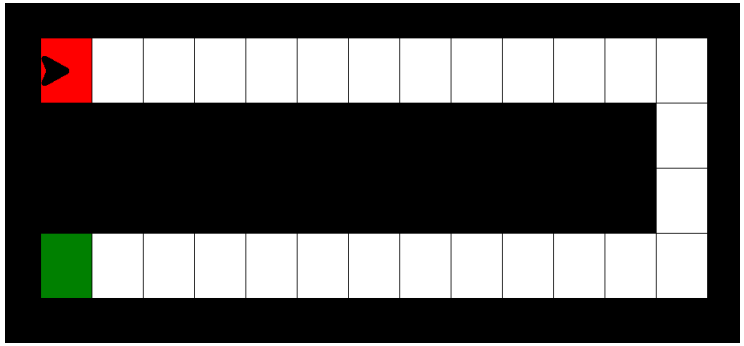
```
while freeForward():
    goForward()
turnRight()
...
```

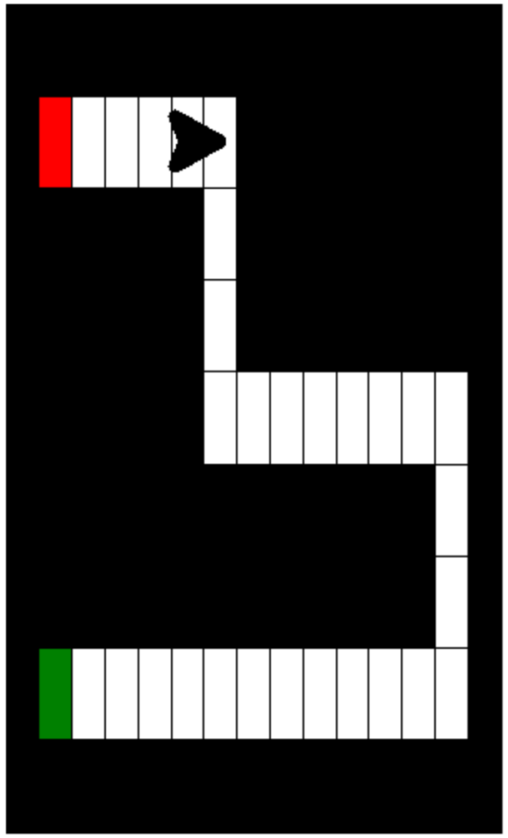
Repeated part done? Stop indentation! Use <Backspace>



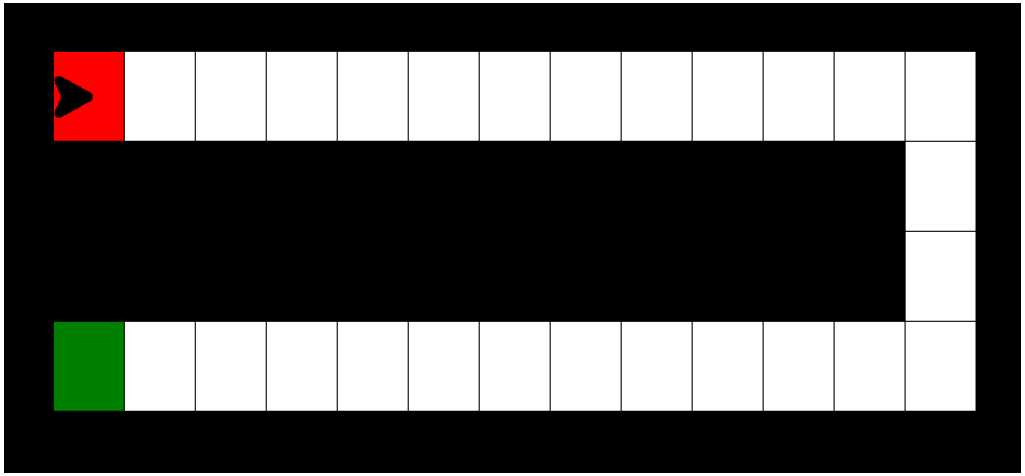
Assignments 2a & 2b

Load the maze and go to the target tile. Use a loop!



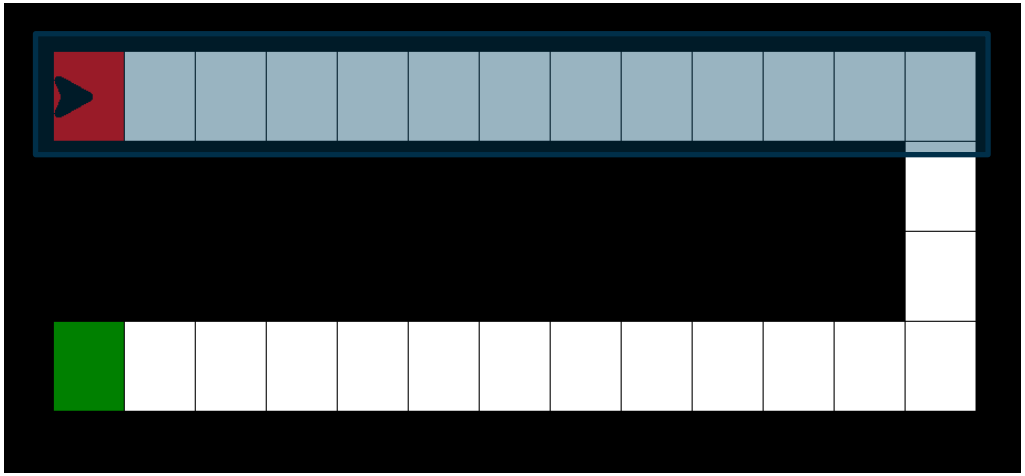


Solution #2a



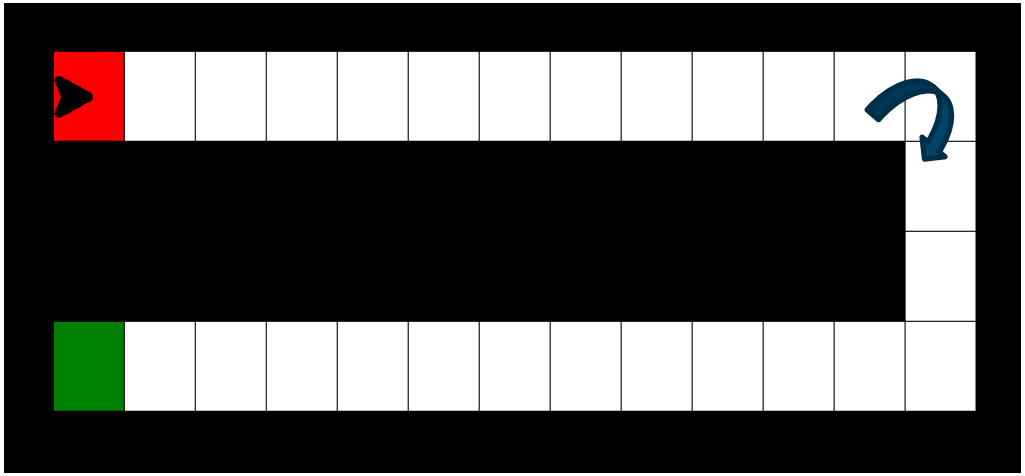
```
# Question 2a: Solve maze with while
loadMaze("maze2a.txt")

while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
```

Solution #2a



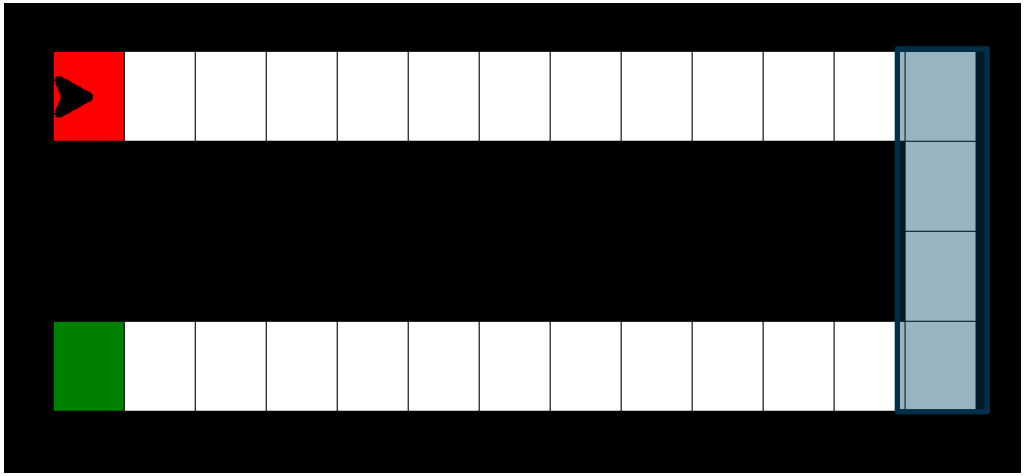
```
# Question 2a: Solve maze with while
loadMaze("maze2a.txt")

while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
```

Solution #2a



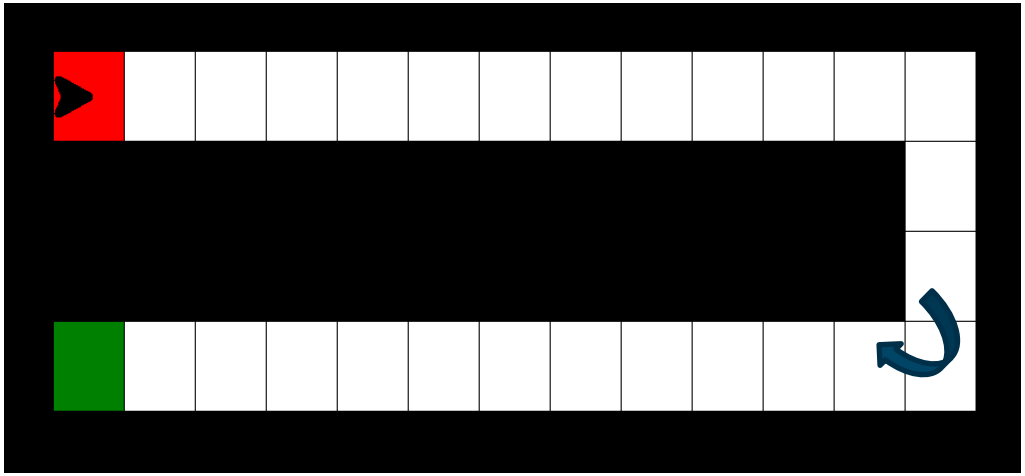
```
# Question 2a: Solve maze with while
loadMaze("maze2a.txt")

while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
```

Solution #2a



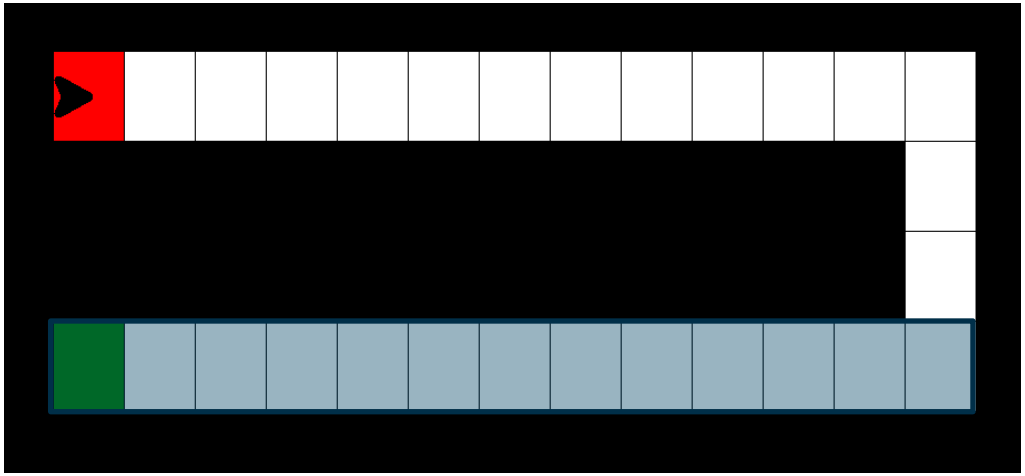
```
# Question 2a: Solve maze with while
loadMaze("maze2a.txt")

while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
```

Solution #2a



```
# Question 2a: Solve maze with while
loadMaze("maze2a.txt")

while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
```

Solution #2a



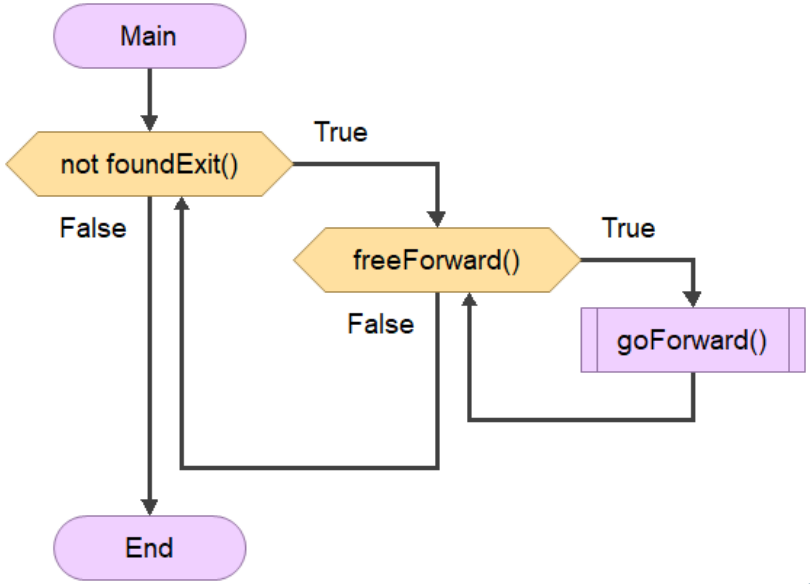
```
# Question 2a: Solve maze with while
loadMaze("maze2a.txt")

while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
```

Solution #2a : less code!

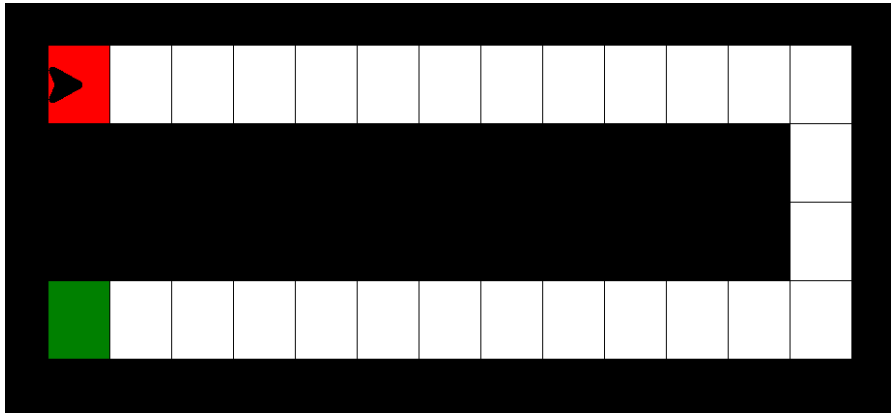
```
# Question 2a: Solve maze with while
loadMaze("maze2a.txt")

while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
turnRight()
while freeForward():
    goForward()
```

+ turnRight()

```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

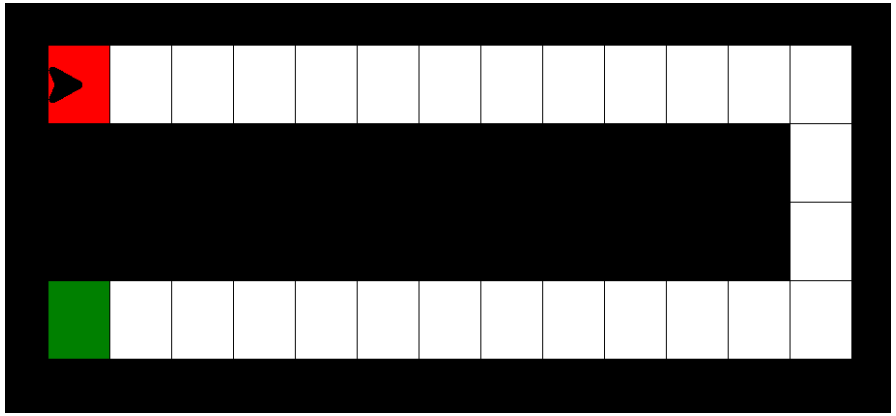


Solution #2a : less code



```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

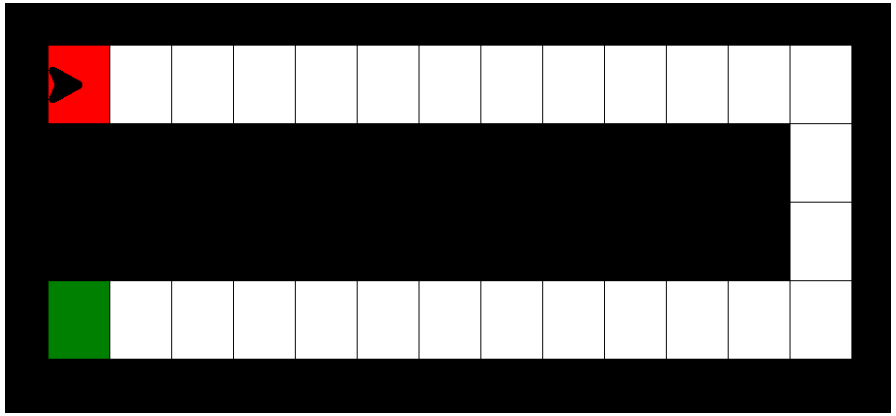
Solution #2a : less code



?

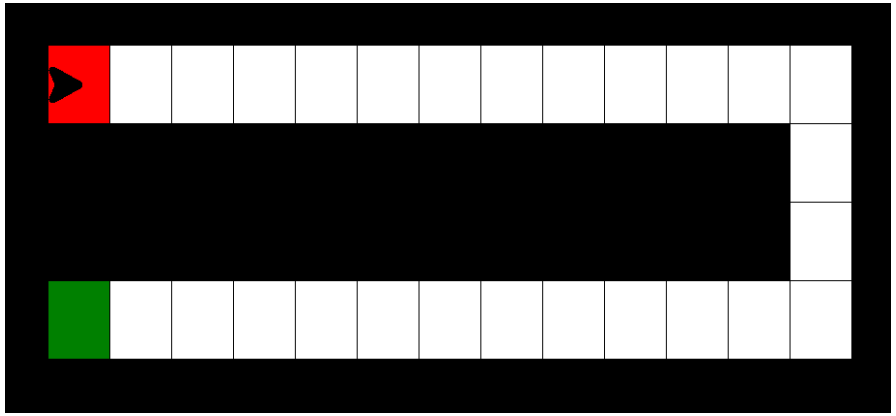
```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

Solution #2a : less code



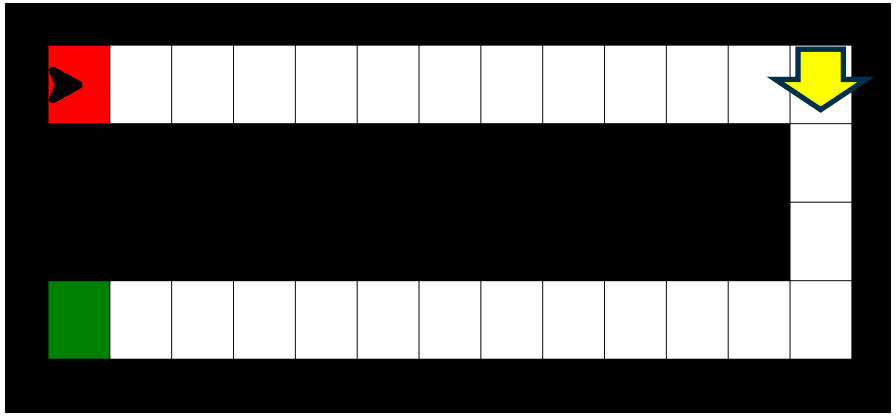
```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

Solution #2a : less code



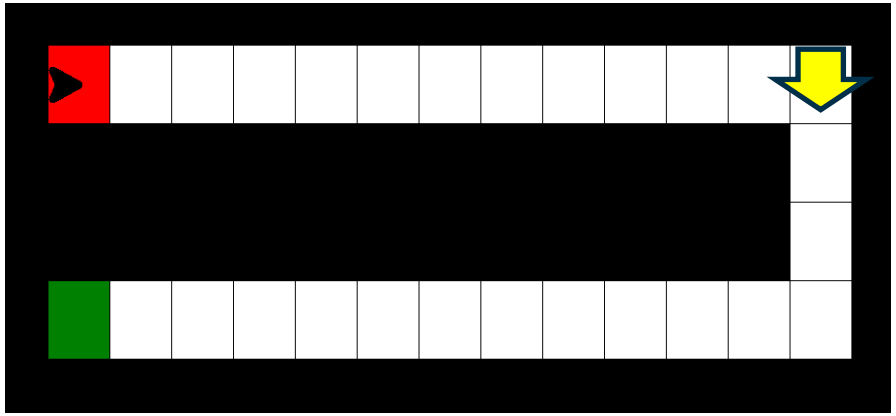
```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

Solution #2a : less code



```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

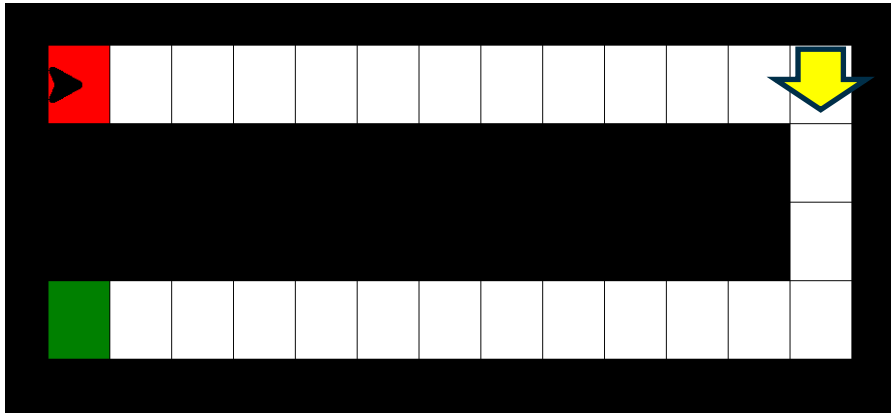
Solution #2a : less code



?

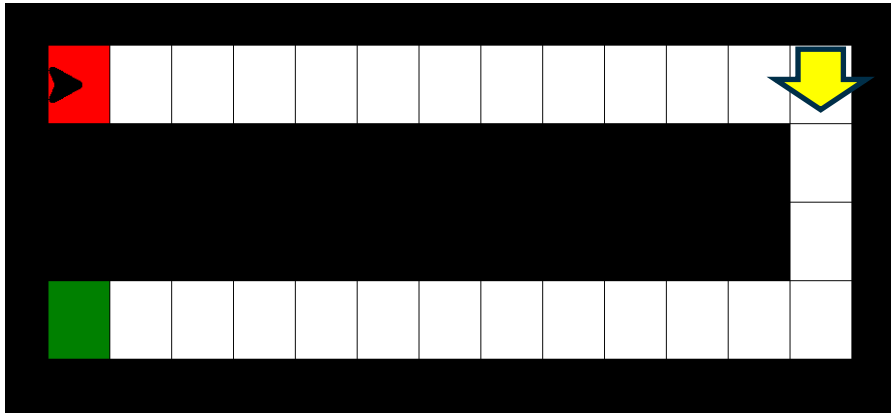
```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

Solution #2a : less code



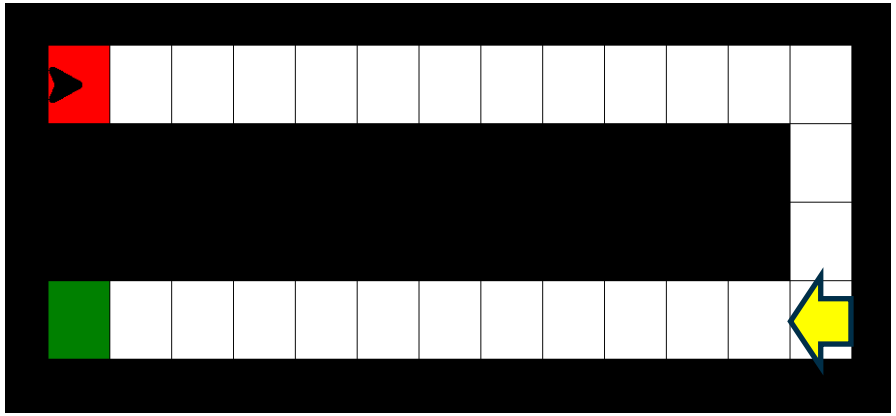
```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

Solution #2a : less code



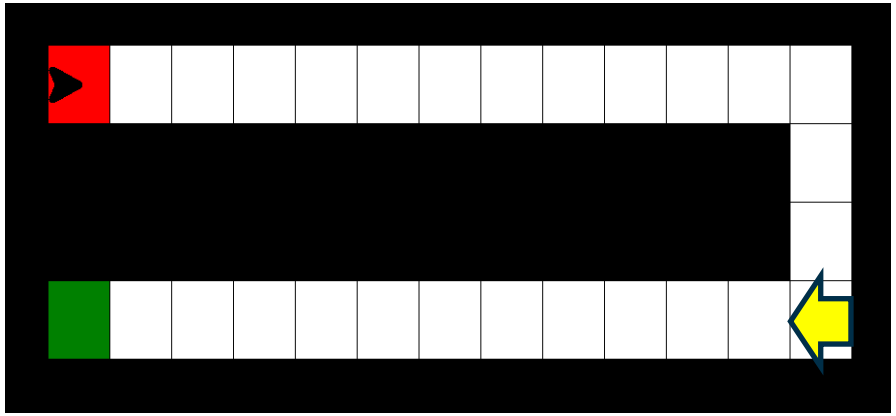
```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

Solution #2a : less code



```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

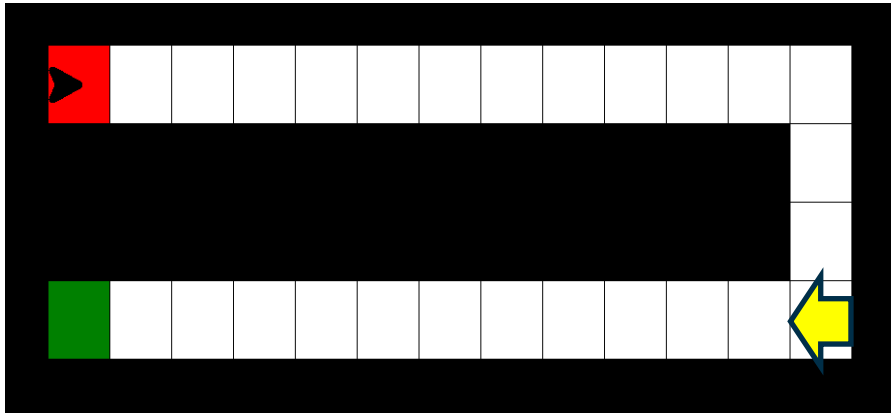
Solution #2a : less code



?

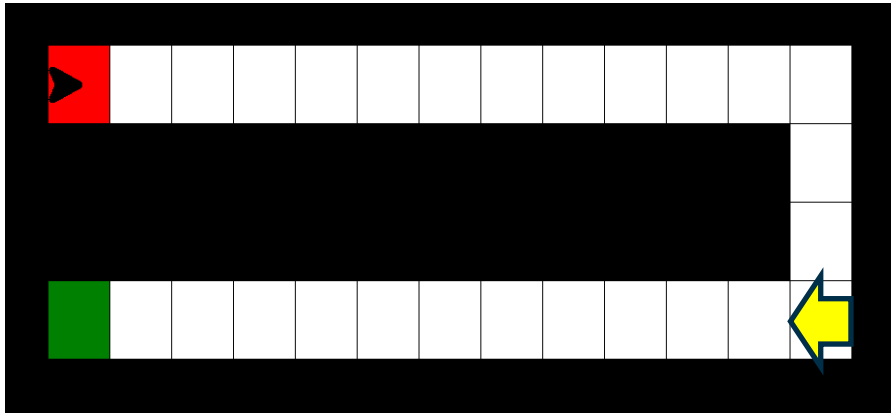
```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

Solution #2a : less code



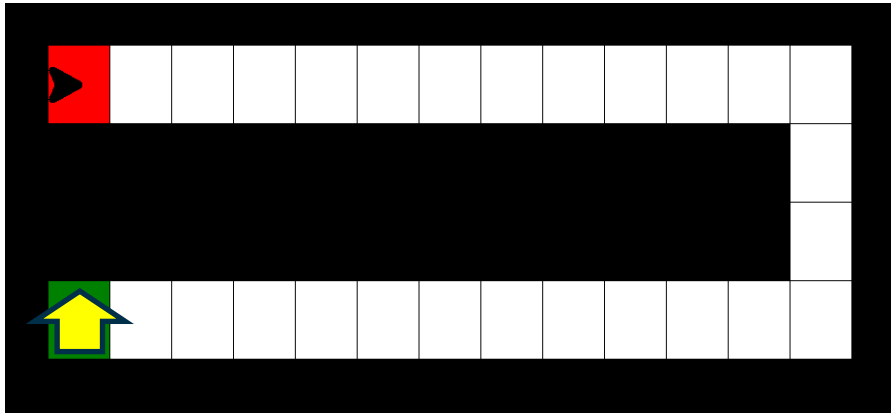
```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

Solution #2a : less code



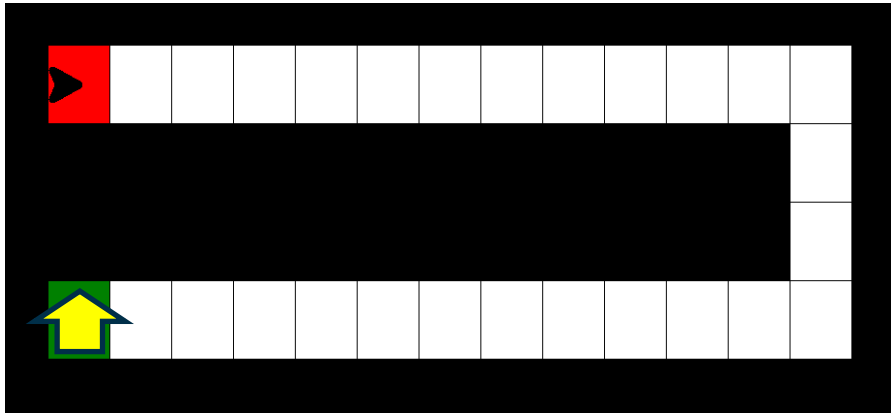
```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

Solution #2a : less code



```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

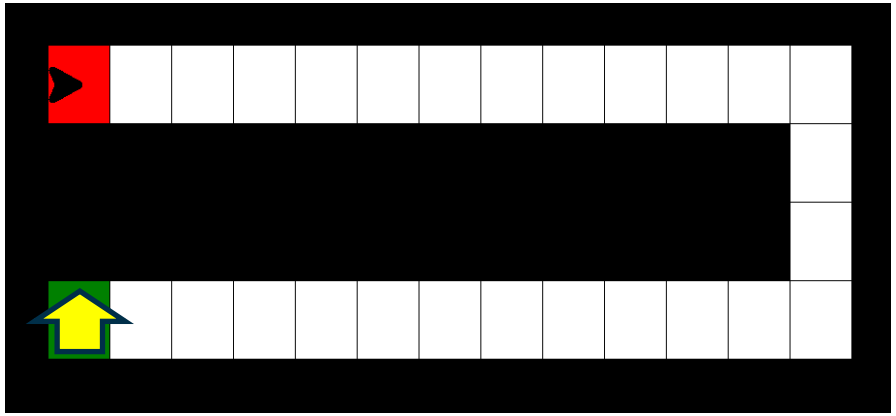
Solution #2a : less code



?

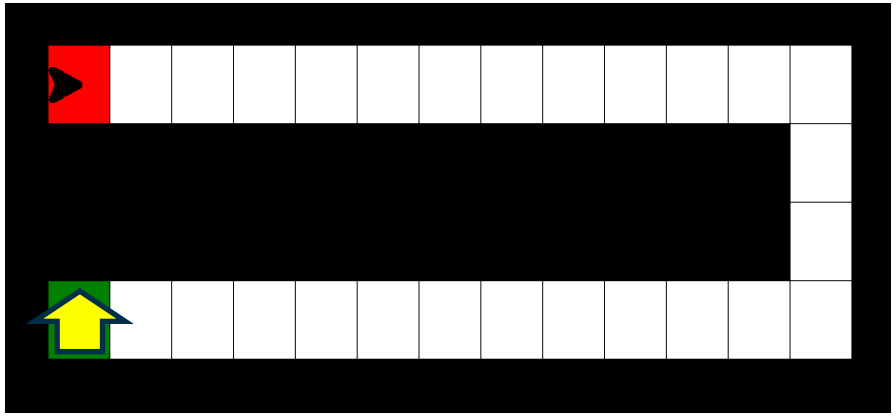
```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

Solution #2a : less code



```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

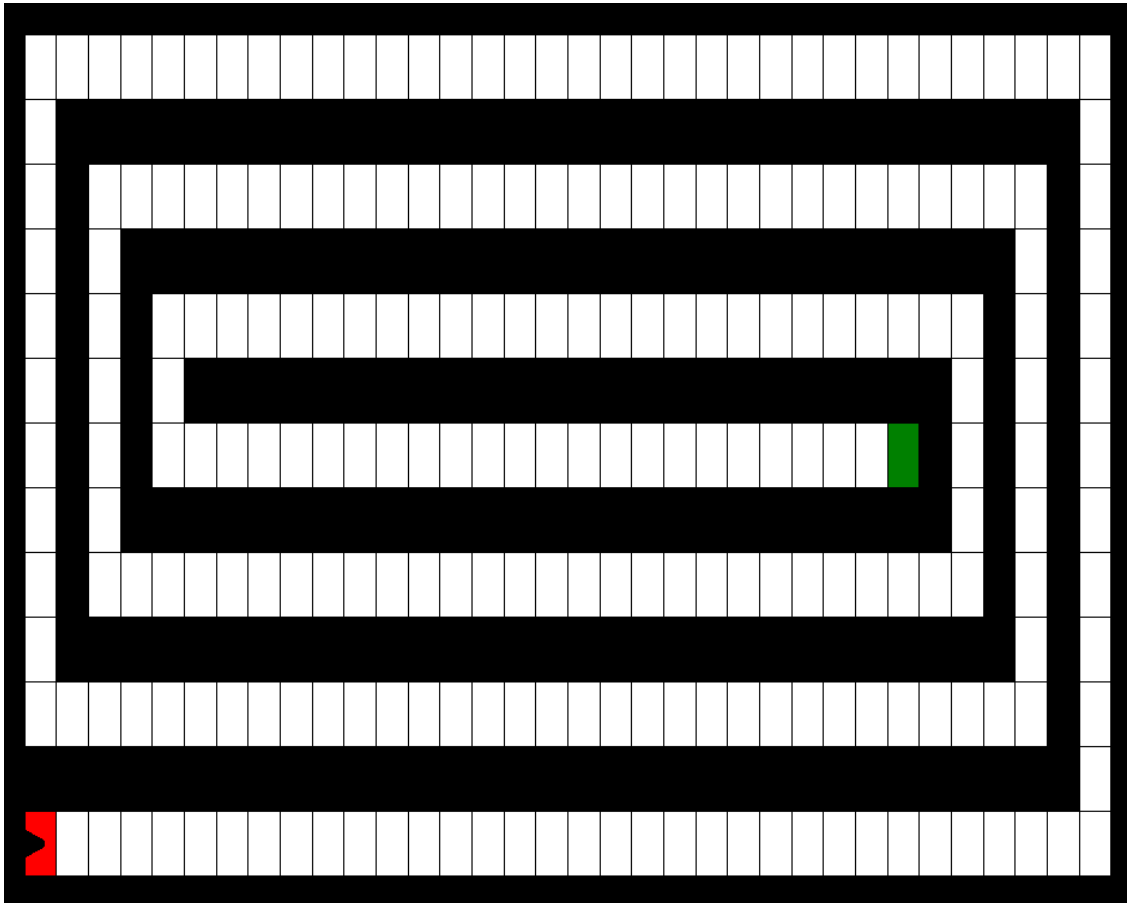
Solution #2a : less code



```
while not foundExit():
    while freeForward():
        goForward()
    turnRight()
```

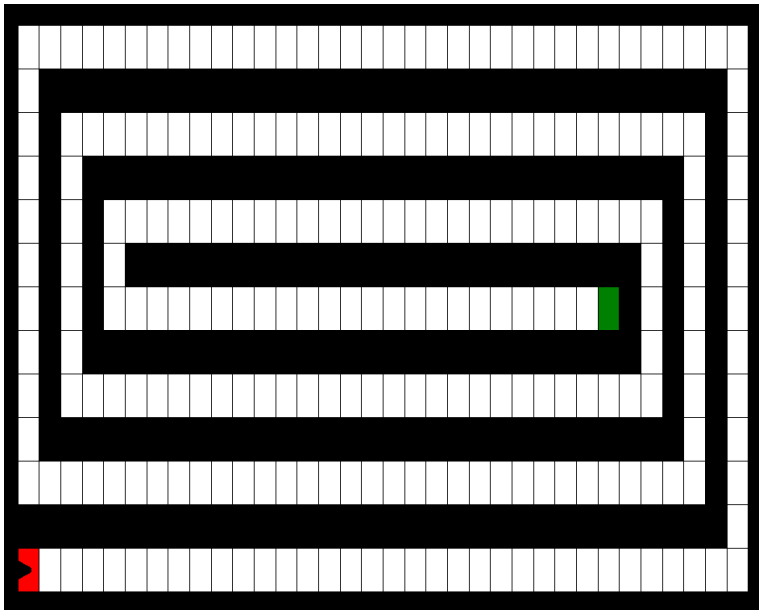
Assignment #2c (q2c.py)

Go to the target tile in the following maze:



Solution #2c

```
while not foundExit():
    while freeForward():
        goForward()
    turnLeft()
```



Solving Arbitrary Mazes

Until now: only programs for solving specific mazes; the solution for Q1 doesn't work for Q2 and vice versa.

Can we write programs that are flexible enough to solve any maze ?

- Implies that we need to make choices depending on the situation:

  If left is free, then turn left

  else if right is free, then turn right

  else go forward

if – elif - else

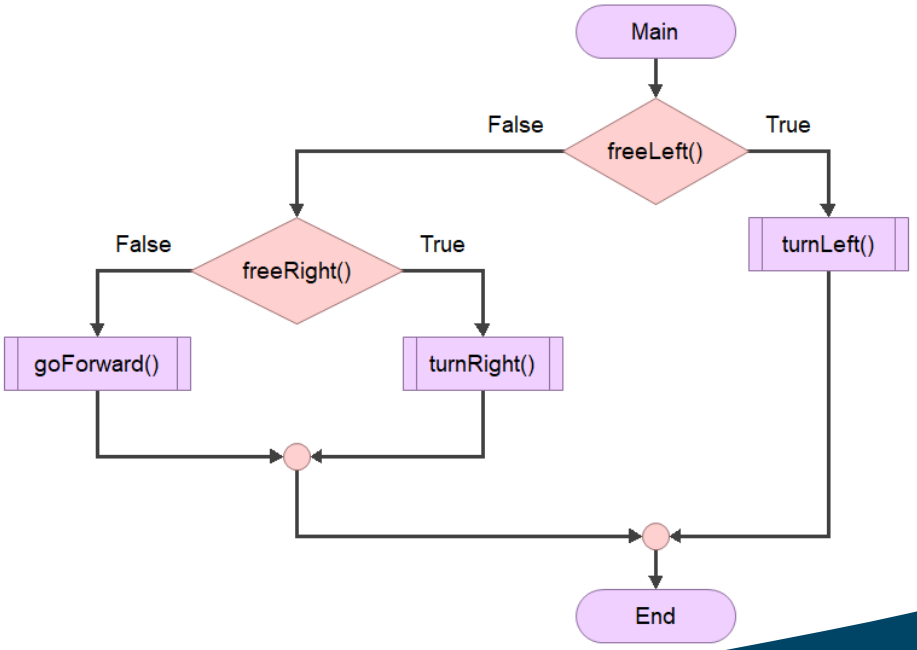
This is exactly what if – elif – else does in Python:

If left is free, then turn left

else if right is free, then turn right

else go forward

```
if freeLeft():
    turnLeft()
elif freeRight():
    turnRight()
else:
    goForward()
```



if – elif - else

This is exactly what if – elif – else does in Python:

lowercase

Do not forget ":"

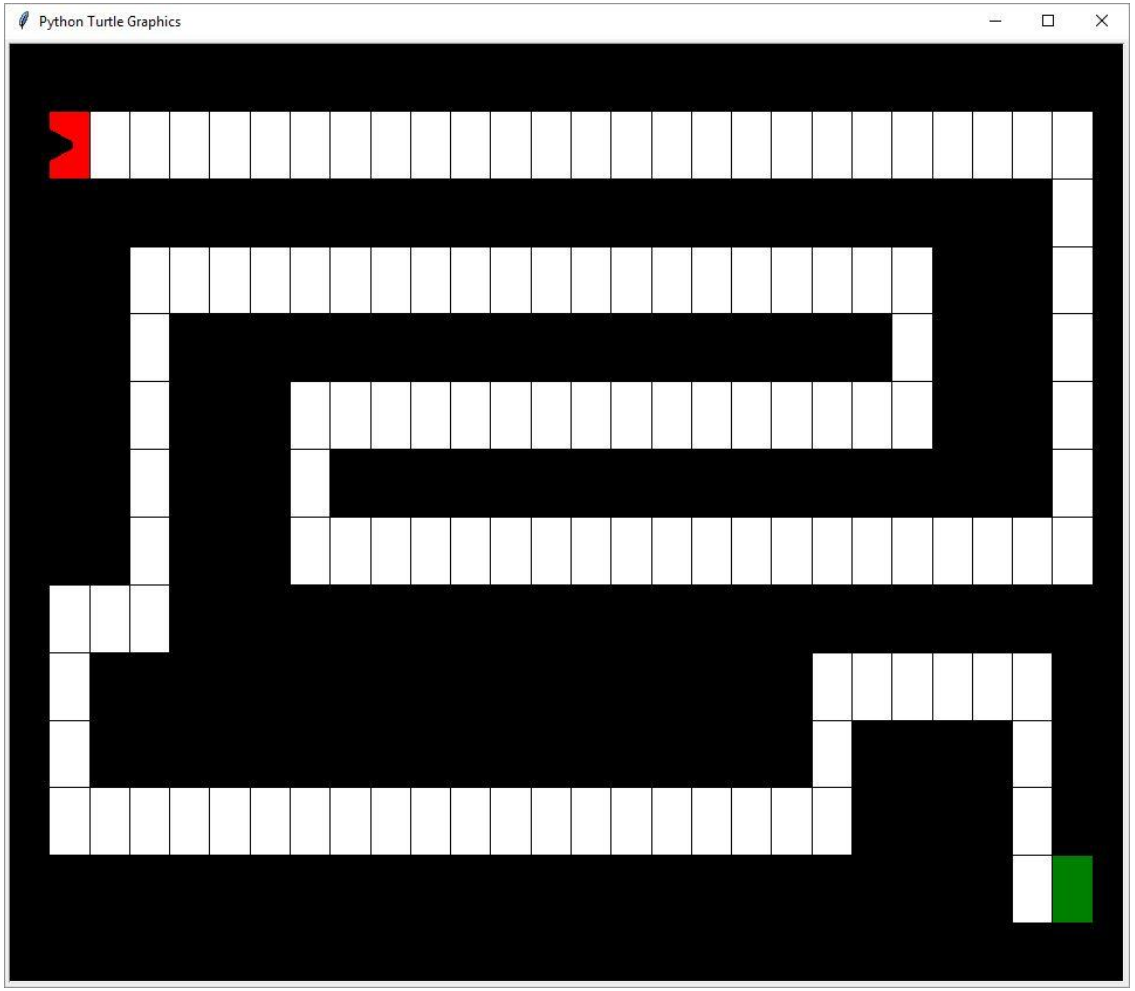
```
if freeLeft():
    turnLeft()
elif freeRight():
    turnRight()
else:
    goForward()
```

if, elif, or else done? unindent (use <Backspace>)

The conditional parts are indented; use <TAB>

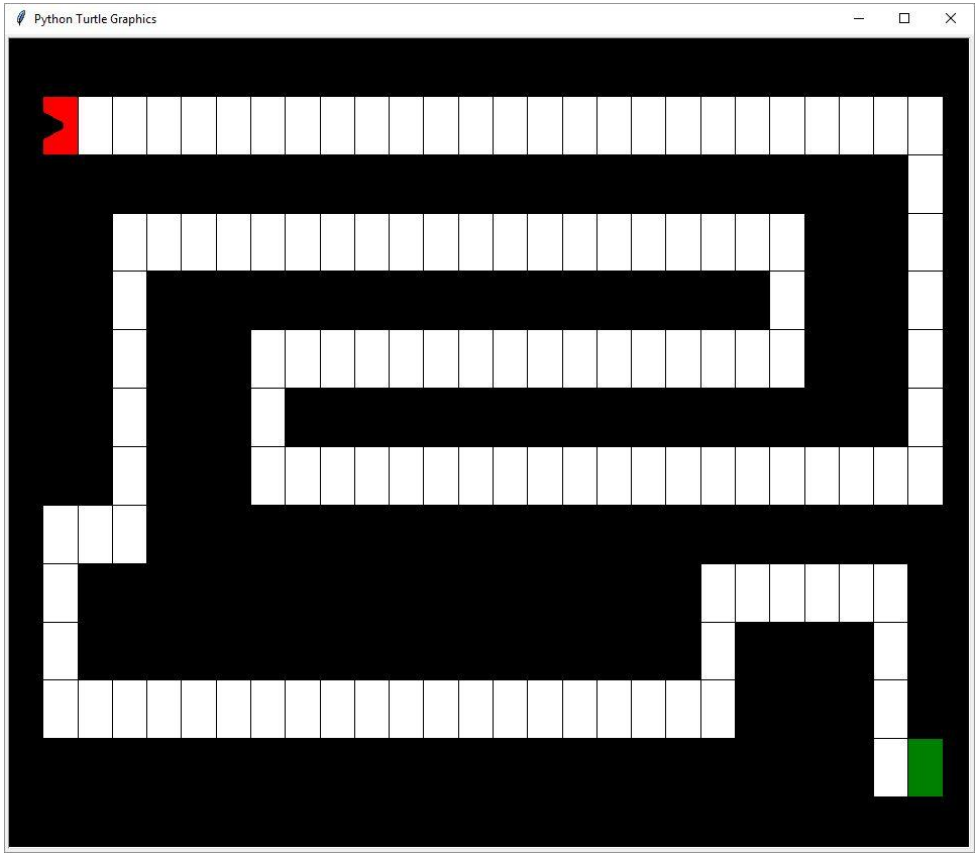
Assignment #3a q3a.py

Solve maze number 3a:



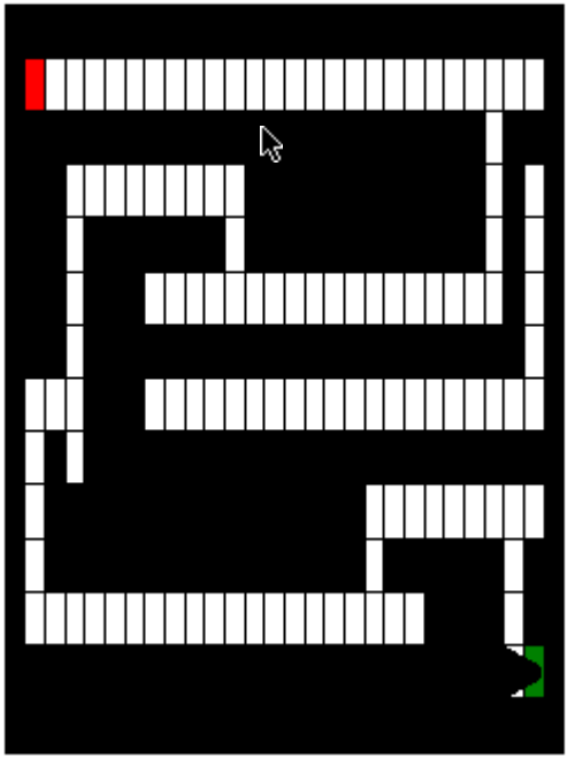
Solution #3a

```
while not foundExit():
    while freeForward():
        goForward()
    if freeLeft():
        turnLeft()
    else:
        turnRight()
```



Assignment #3b q3b.py

Solve maze number 3b:



Hint:

Conditions can be combined with and, or, not

```
while not freeRight() and not freeLeft():
    ...
```

The next slides are extra for those interested in a generic solution for any maze*

* with entrance and exit at the exterior wall

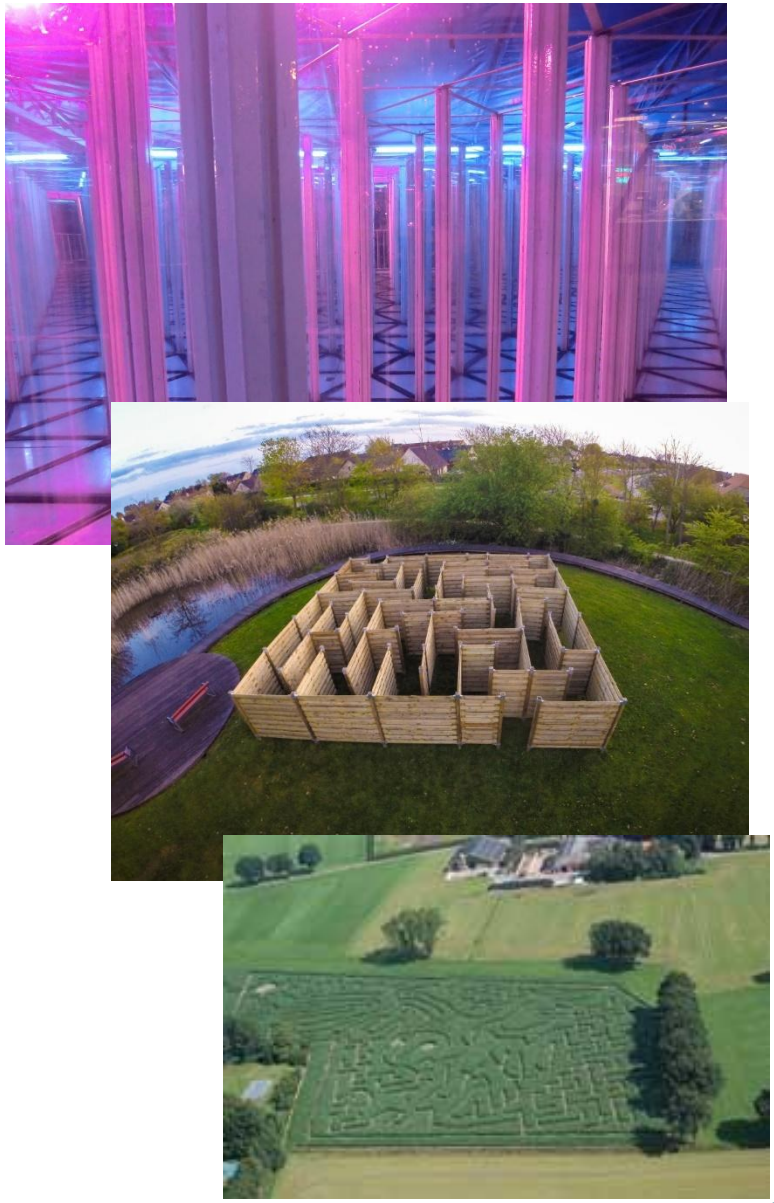
To summary slide

Righthand rule

If start and end positions in the maze are both next to the exterior wall, the following algorithm always finds a path to the target position:

Always follow the rightmost path

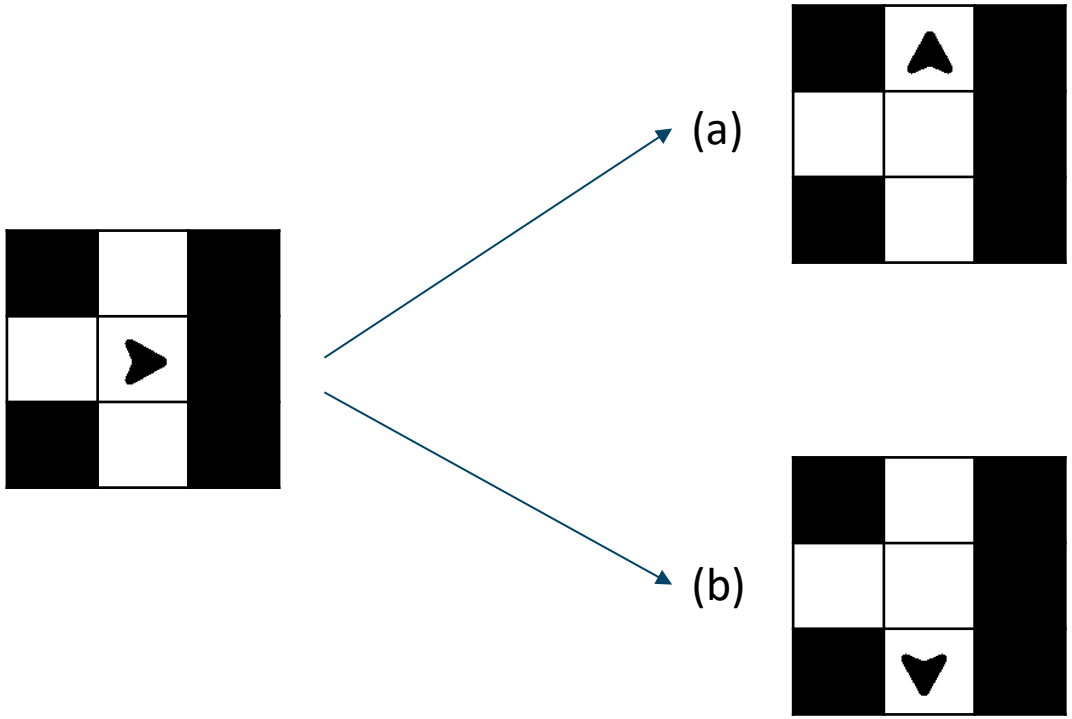
(Make sure your right hand always touches the wall)



Righthand rule

Follow the rightmost path

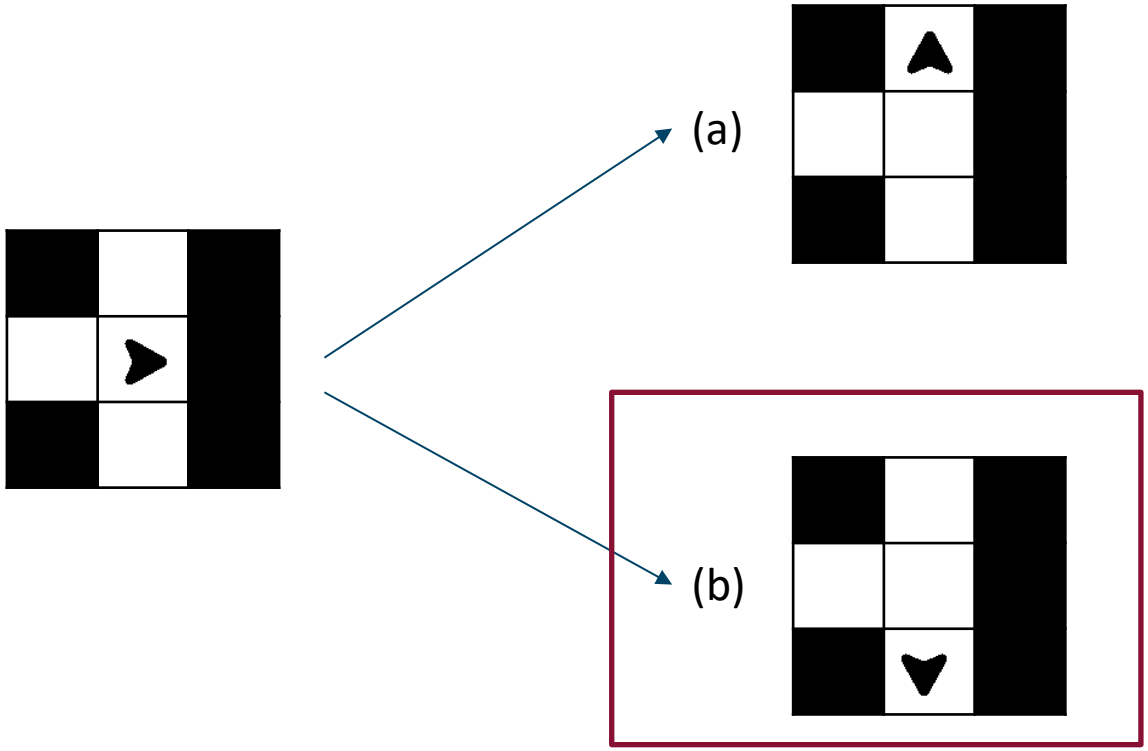
Where to go next in this example?



Righthand rule

Follow the rightmost path

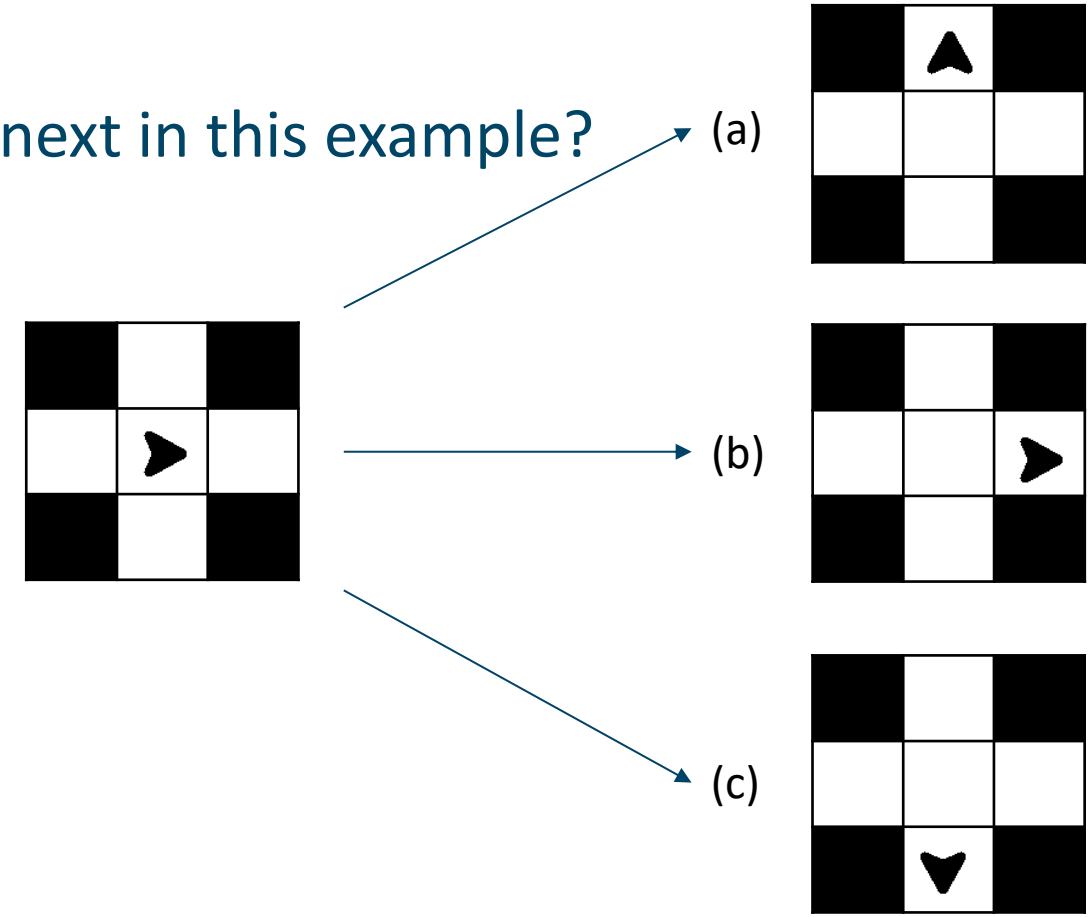
Where to go next in this example?



Righthand rule

Follow the rightmost path

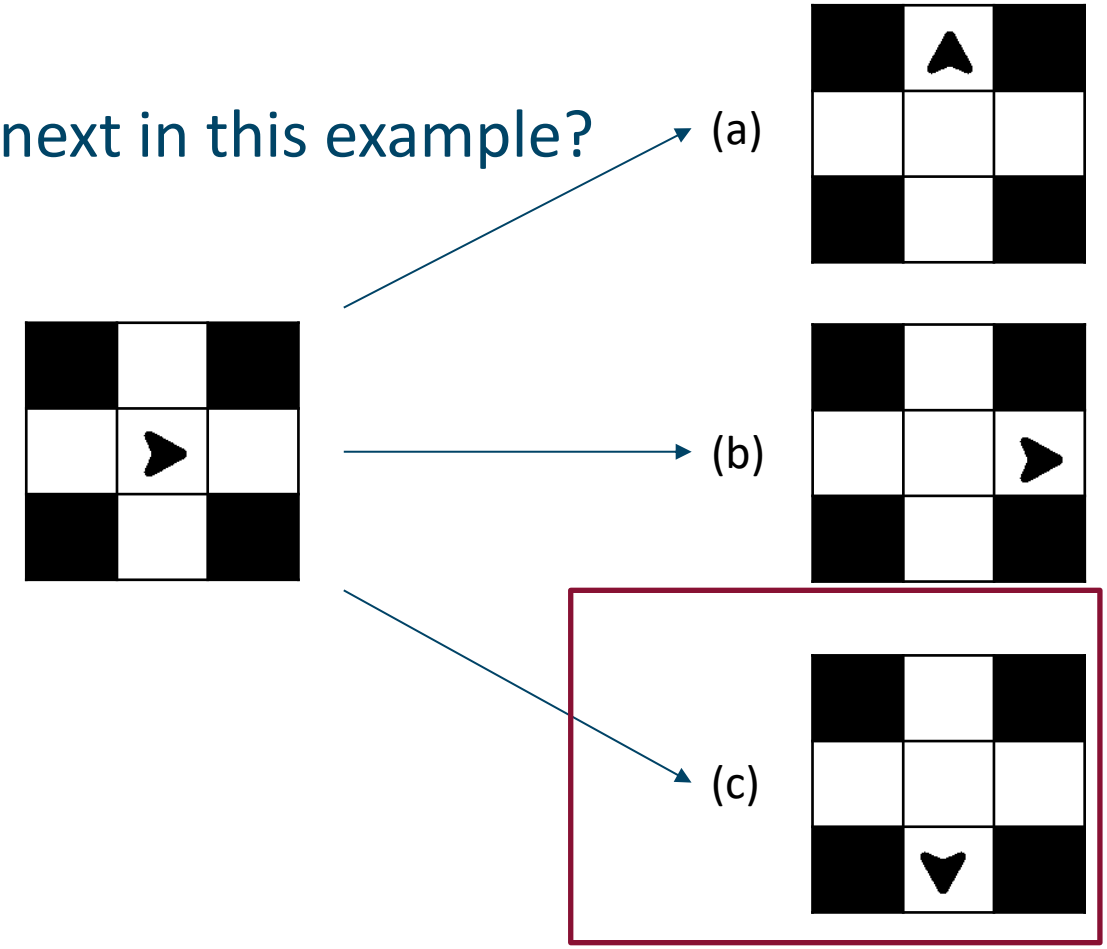
Where to go next in this example?



Righthand rule

Follow the rightmost path

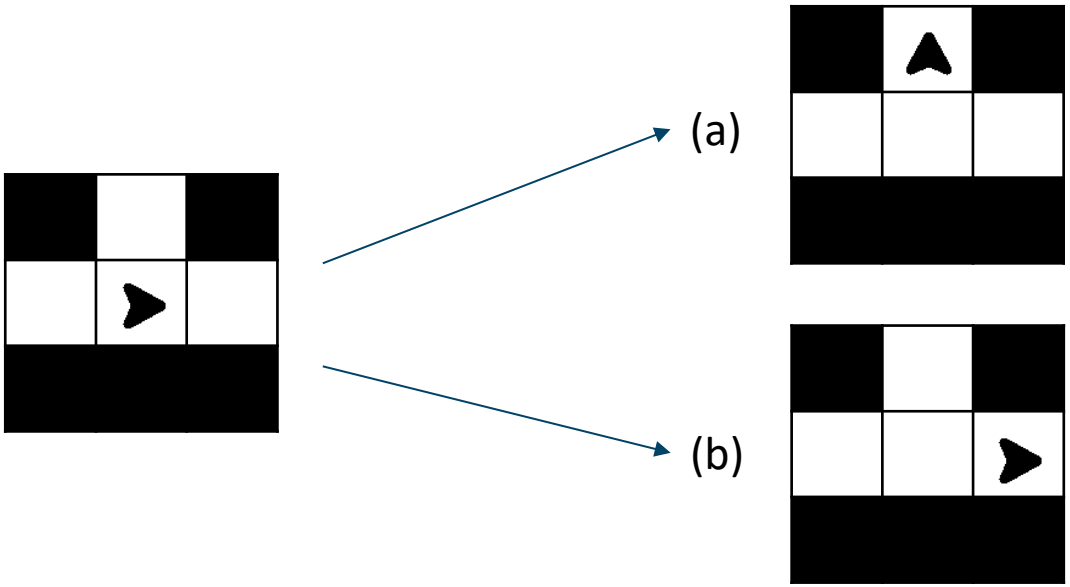
Where to go next in this example?



Righthand rule

Follow the rightmost path

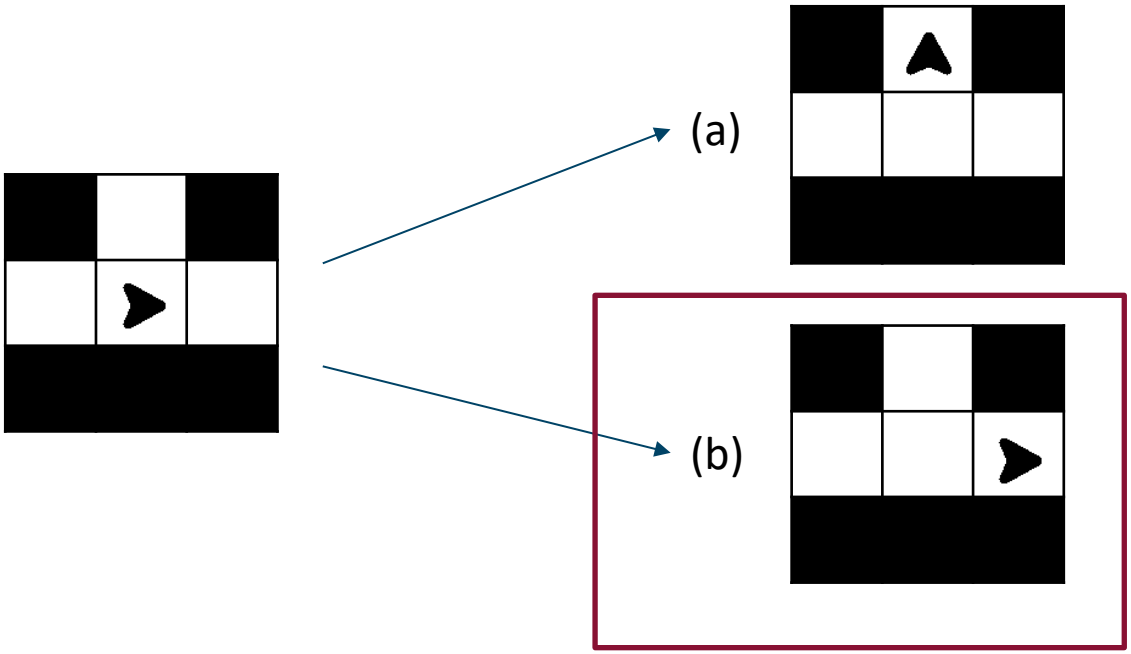
Where to go next in this example?



Righthand rule

Follow the rightmost path

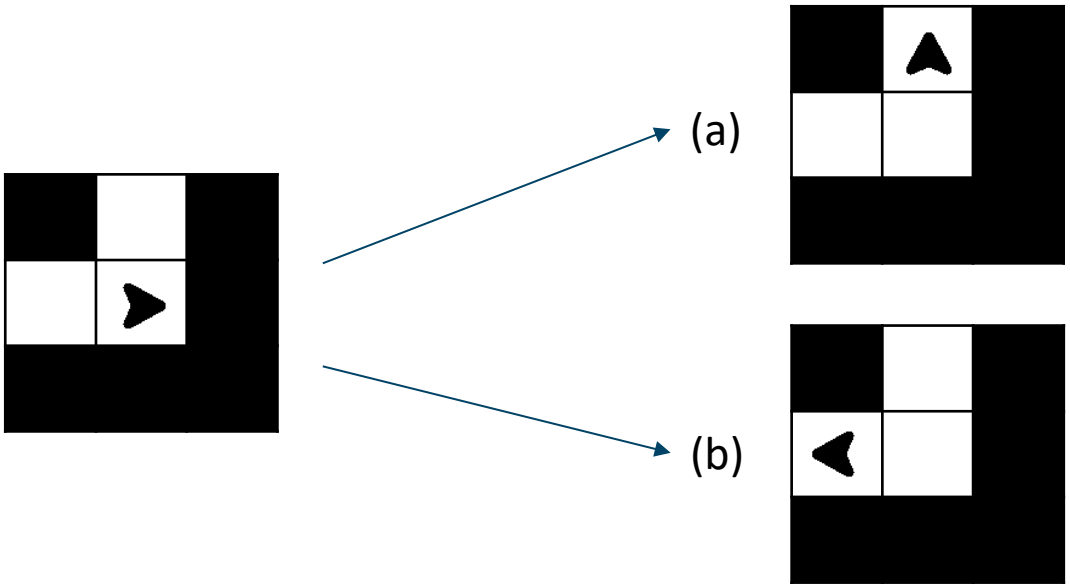
Where to go next in this example?



Righthand rule

Follow the rightmost path

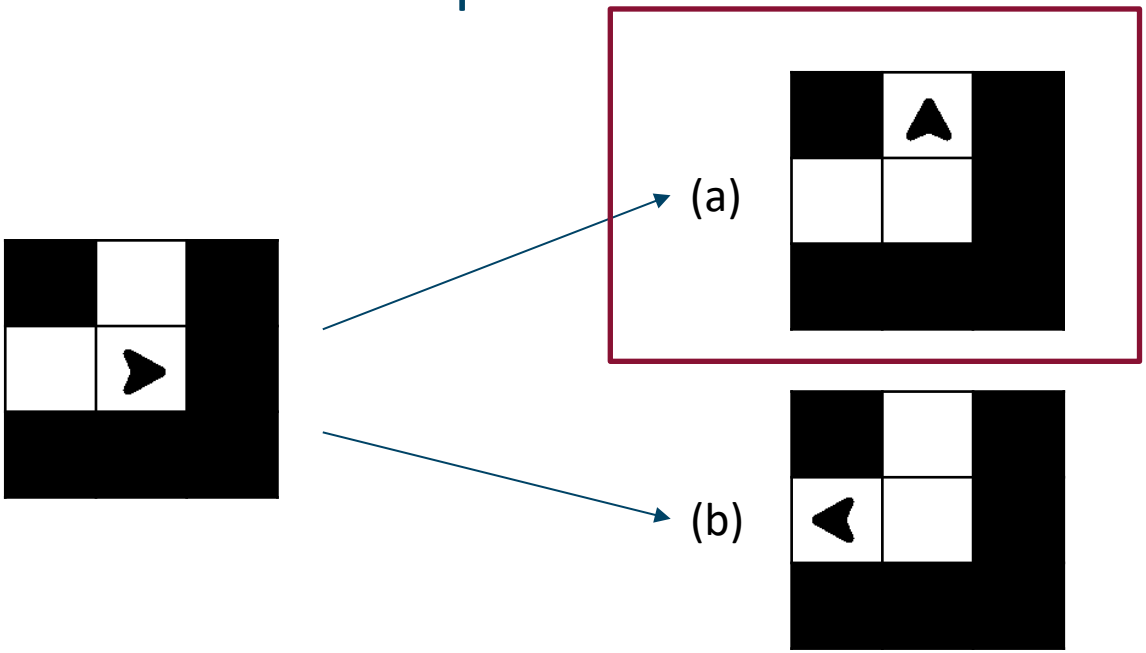
Where to go next in this example?



Righthand rule

Follow the rightmost path

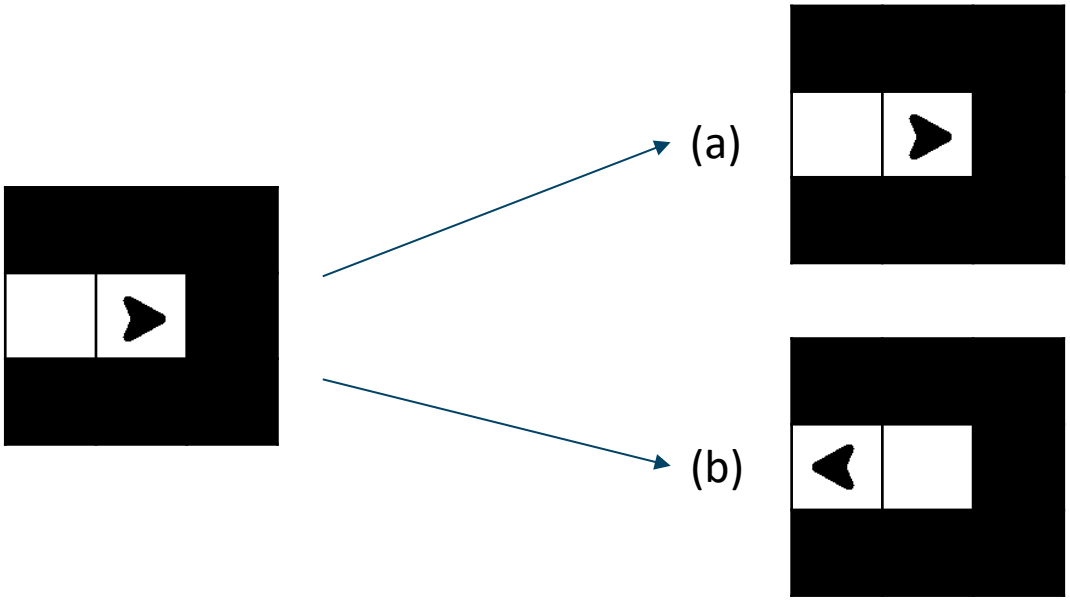
Where to go next in this example?



Righthand rule

Follow the rightmost path

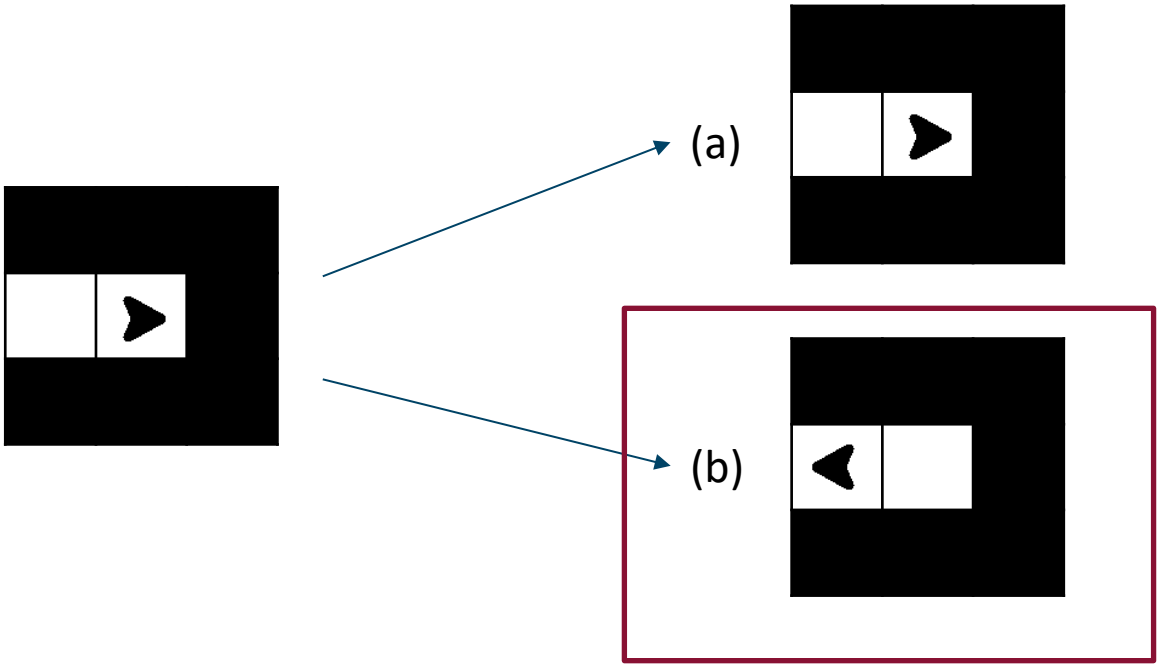
Where to go next in this example?



Righthand rule

Follow the rightmost path

Where to go next in this example?



Righthand rule

Always follow the rightmost path

=

Until you reach the exit, repeat:

if right is free:

turn right and go forward

else if forward is free:

go forward

else if left is free is:

turn left and go forward

else:

turn 180 degrees and go forward

Assignment #4 - **together** q4.py

Implement the righthand rule. You can test on maze #4.

```
# Question 3: Solve maze4

loadMaze("maze4.txt")     # Remove the # in front of this rule to load maze 4
```

Always follow the rightmost path, until you reach the end

Finished? Try also maze #5!

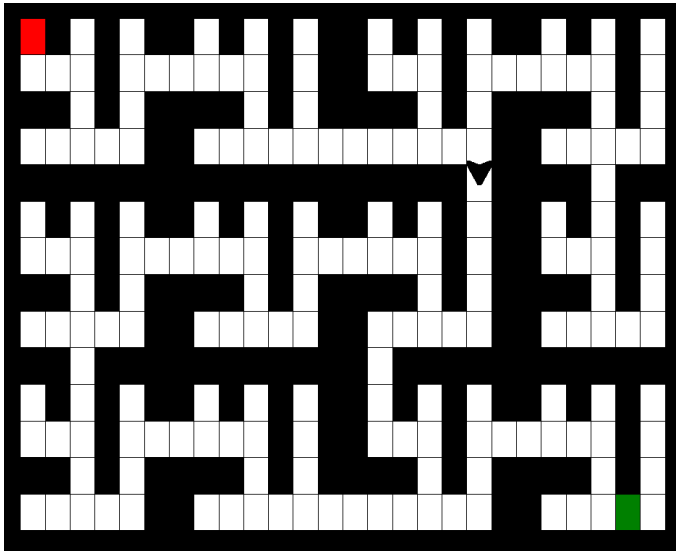
Solution #4

As long as you haven't reached the exit:

Search the rightmost free next position

go forward

```
while(not foundExit()):
    if freeRight():
        turnRight()
        goForward()
    elif freeForward():
        goForward()
    elif freeLeft():
        turnLeft()
        goForward()
    else:
        turnRight()
        turnRight()
        goForward()
```



Summary Part 1

Programming = give step-wise instructions to solve a problem.

Important constructions:

Sequence of commands : execute one by one

Repetition loops : while … repeat …

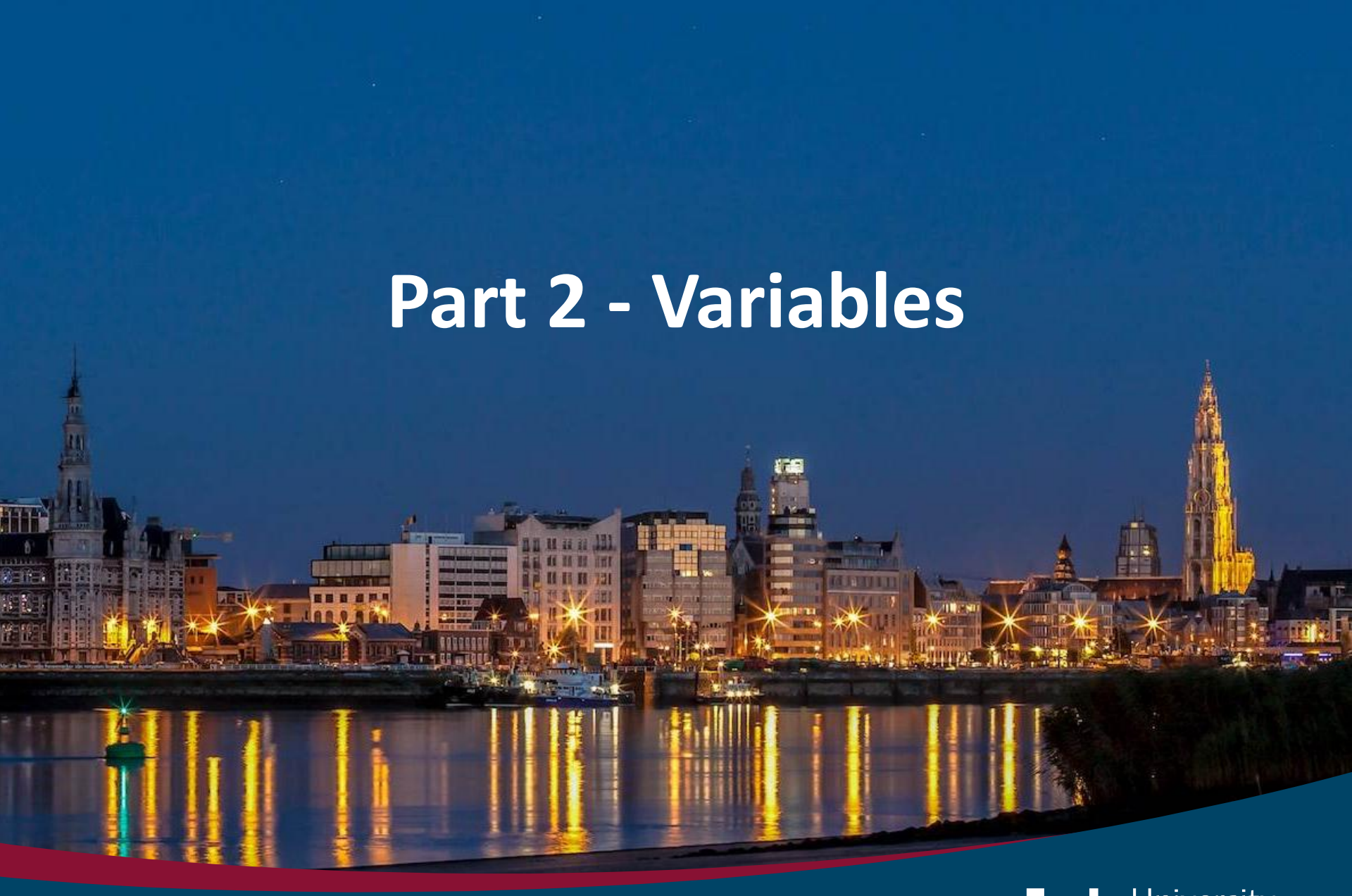
Conditional execution : if … elif … else …

One important concept is still missing:

Variables to temporarily store things in memory

University of Antwerp

Part 2 - Variables



University of Antwerp

Part 2 - Variables

Variable : place to store a value in memory

```
1  name="Toon"
2  print("Hello, my name is",name)
```

Variables - Example

Ask the average grade, tell Python it is a real number (float)

```
avg=float(input("Average grade ? "))
if avg < 10: ?
    grade="Fail"
elif avg < 14: ?
    grade="Pass"
elif avg < 15: ?
    grade="Distinction"
elif avg <17:  ?
    grade="Cum Laude"
else:  # average >= 17
    grade="Magna Cum Laude"

print(grade)
```

15

Variable grade is made and given value “Cum Laude”

Cum Laude

Exercise 5 – Generation

Ask birthyear *hint: a number is "int" → y=int(input(…*

Categorize into:

- The Baby Boomer Generation – born 1946-1964.
- Generation X – born 1965-1979.
- Millennials – born 1980-1994.
- Generation Z – born 1995-2012.
- Gen Alpha – born 2013 – 2025.

Print the generation someone born in that year belongs to

Example : interest rate

```
loan=float(input("give loan amount: "))
interest=float(input("give monthly interest rate: "))
payment=float(input("monthly payment: "))
print("outstanding loan after 1 month",loan*(1+interest)-payment)
```

10000

0.01

150

9950.0

Variable	value

Reference – operations

operation	meaning	example	result
+	addition	3 + 6	9
-	subtraction	3.0 - 7	-4.0
*	multiplication	1.7 * 100	170.0
/	division	9/2	4.5
//	int division	9//2	4
%	remainder	9%2	1
==	equality	2 == 1+1	True
!=	inequality	2 != 3	True
>	more	5+3 > 6+2	False
<	less	5 < 6	True
<=	less or equal	3<=2	False
>=	more or equal	5+3 >= 6+2	True

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Execute for month ranging from 1 to (not including) 10

Variable	value
loan	10000.0
interest	0.1
payment	150.0

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	10000.0

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	10000.0
month	1

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	10000.0
month	1

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	10850.0
month	1

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	10850.0
month	1

```
Outstanding loan after 1 months: 10850.0
```

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	10850.0
month	2

```
Outstanding loan after 1 months: 10850.0
```

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	10850.0
month	2

```
Outstanding loan after 1 months: 10850.0
```

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	11785.0
month	2

```
Outstanding loan after 1 months: 10850.0
```

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	11785.0
month	2

```
Outstanding loan after 1 months: 10850.0
Outstanding loan after 2 months: 11785.0
```

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	11785.0
month	3

```
Outstanding loan after 1 months: 10850.0
Outstanding loan after 2 months: 11785.0
```

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	11785.0
month	3

```
Outstanding loan after 1 months: 10850.0
Outstanding loan after 2 months: 11785.0
```

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	12813.5
month	3

```
Outstanding loan after 1 months: 10850.0
Outstanding loan after 2 months: 11785.0
```

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	12813.5
month	3

```
Outstanding loan after 1 months: 10850.0
Outstanding loan after 2 months: 11785.0
Outstanding loan after 3 months: 12813.5
```

...

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	21542.55…
month	9

```
Outstanding loan after 1 months: 10850.0
Outstanding loan after 2 months: 11785.0
Outstanding loan after 3 months: 12813.5
…
Outstanding loan after 9 months: 21542.6
```

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	21542.55…
month	10

```
Outstanding loan after 1 months: 10850.0
Outstanding loan after 2 months: 11785.0
Outstanding loan after 3 months: 12813.5
…
Outstanding loan after 9 months: 21542.6
```

Payment Table

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Variable	value
loan	10000.0
interest	0.1
payment	150.0
outstanding	21542.55…
month	10

```
Outstanding loan after 1 months: 10850.0
Outstanding loan after 2 months: 11785.0
Outstanding loan after 3 months: 12813.5
…
Outstanding loan after 9 months: 21542.6
```

Note: *for* versus *while*

Everything we can do with for we can also do with while:

```
outstanding=loan
for month in range(1,10):
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
```

```
outstanding=loan
month=1
while month<10:
    outstanding=outstanding*(1+interest)-payment
    print("outstanding loan after",month, "months:",outstanding)
    month=month+1
```

Exercise 6 – interest rate

Change the code:

- Ask the user for how many years they want the table
- Print the monthly table for that many years

Extensions:

- Stop if the outstanding amount reaches (or becomes lower than) 0
- Compute the total payment
  - Take into account that the last payment is not for the full amount, but such that the balance is exactly 0

Summary Part 2

Variables to store values

- Different types : int, float, string

Calculations with numbers

- +, -, *, /, //, %

Comparing values

- >, <, ==, !=

Loops that are indexed by variables:

- for variable in range(start,end):

Part 3 - Coding Decision Trees

University of Antwerp

Decision trees for regular use cases

Example

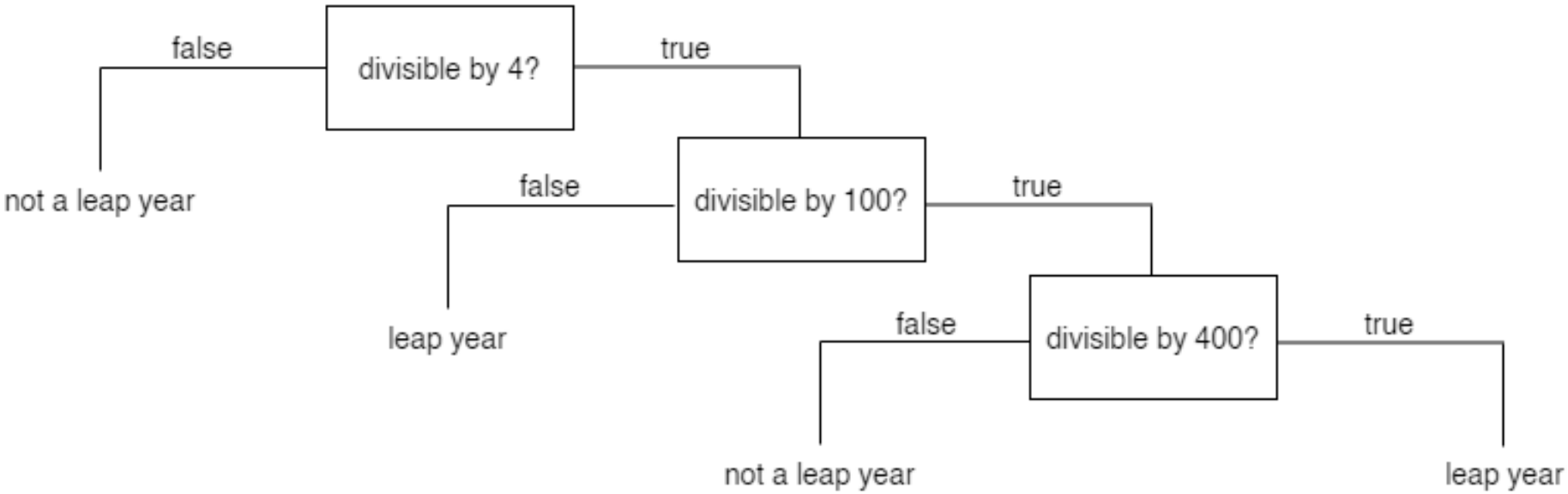
Write a program that checks if a given year is a leap year or not. A leap year is divisible by 4, but not by 100, unless it is divisible by 400. E.g. 2004 and 2000 are leap years, but 1800 and 1900 not.

y%4 : remainder after division by 4

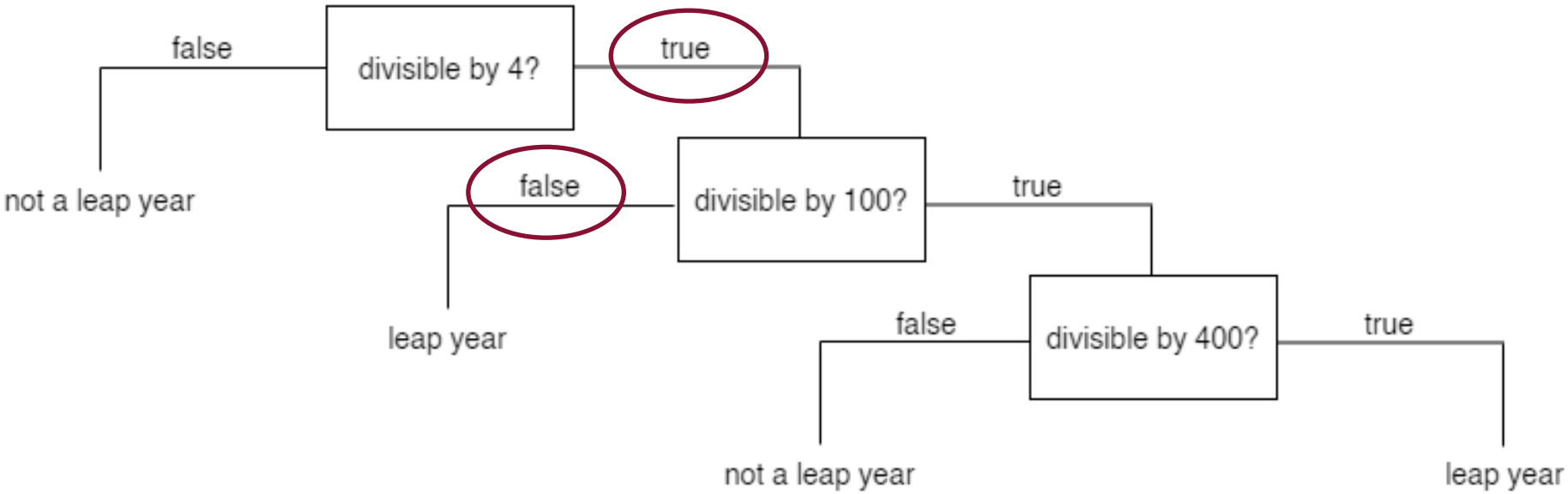
y%4 == 0 : True if y is a multiple of 4

7%4 == 3 → 7 not divisible by 4

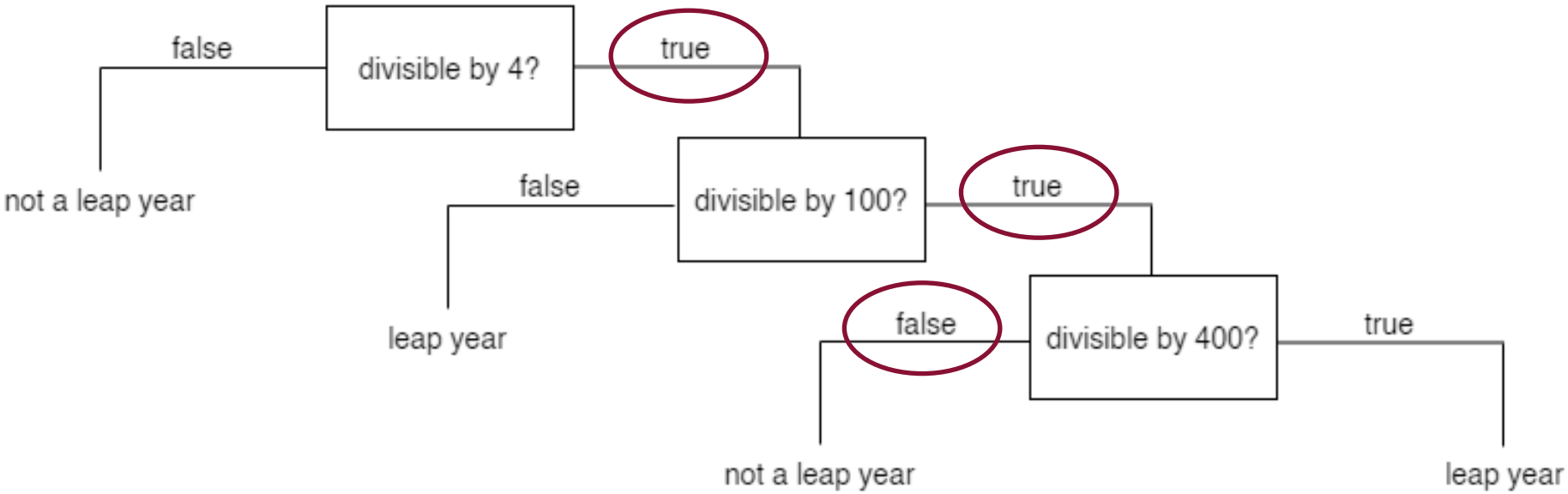
8%4 == 0 → 8 divisible by 4


divisible by 4?
false
true
not a leap year
divisible by 100?
false
true
leap year
divisible by 400?
false
true
not a leap year
leap year

2004


false
divisible by 4?
true
not a leap year
false
divisible by 100?
true
leap year
false
divisible by 400?
true
not a leap year
leap year

1900


false
divisible by 4?
true
not a leap year
false
divisible by 100?
true
leap year
false
divisible by 400?
true
not a leap year
leap year

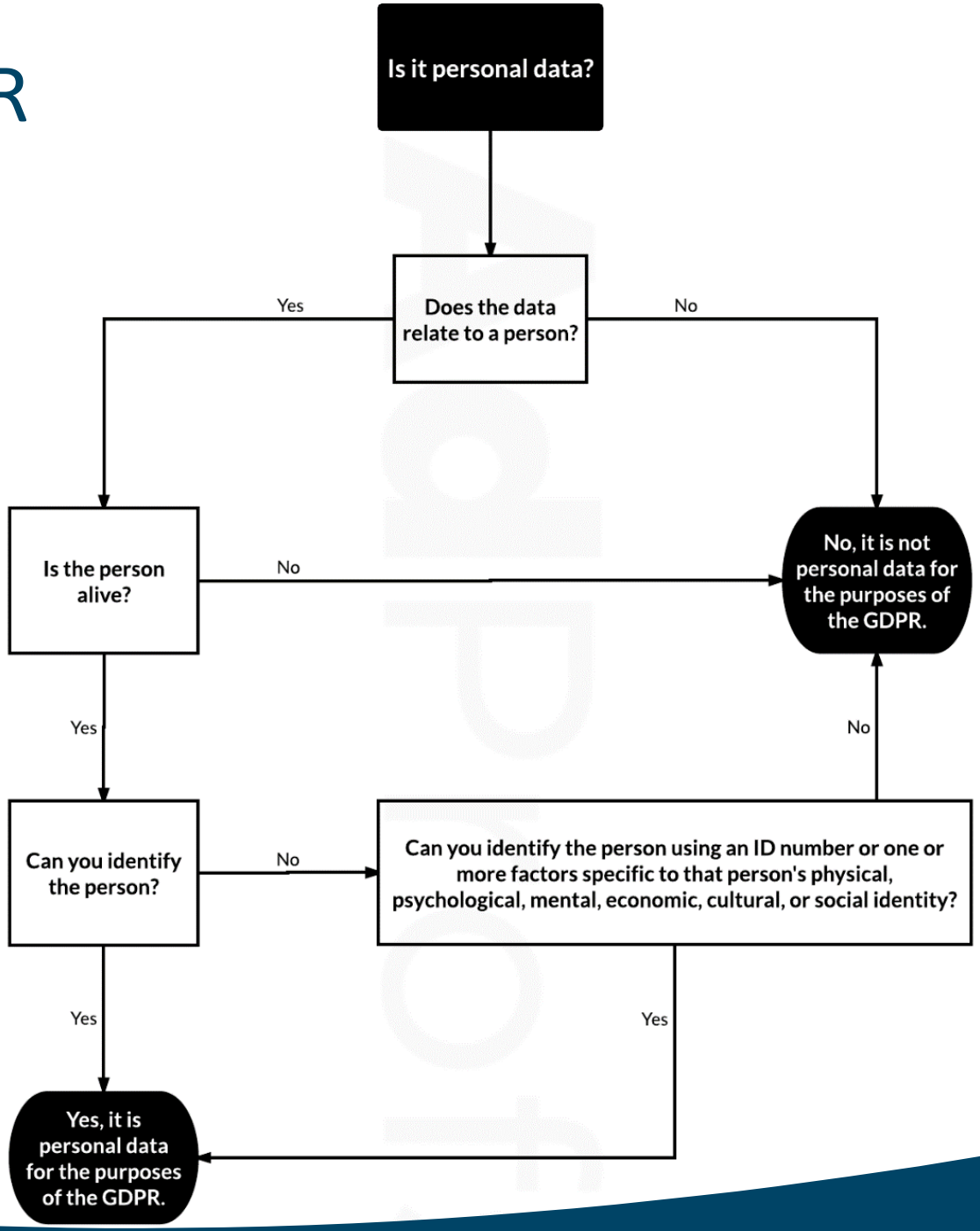
Let’s code!

```
year = int(input("year?"))

if (year % 4 !=0):
    print("not a leap year")
else:
    if (year % 100 != 0):
        print("leap year")
    else:
        if (year % 400 != 0):
            print("not a leap year")
        else:
            print("leap year")
```

Decision trees in law practice

Case 1

Q7: GDPR


Is it personal data?
Does the data relate to a person?
Yes
No
Is the person alive?
No
No, it is not personal data for the purposes of the GDPR.
Yes
No
Can you identify the person?
No
Can you identify the person using an ID number or one or more factors specific to that person's physical, psychological, mental, economic, cultural, or social identity?
Yes
Yes
Yes, it is personal data for the purposes of the GDPR.

Case 2

Q8 - Unfair contract terms

- Directive 93/13 (the Unfair Contract Terms Directive) bans the use of certain terms in contracts between traders and consumers
- Key provisions:

Article 1

1. The purpose of this Directive is to approximate the laws, regulations and administrative provisions of the Member States relating to unfair terms in contracts concluded between a seller or supplier and a consumer.

Article 2

For the purposes of this Directive:

(a) 'unfair terms' means the contractual terms defined in Article 3;

(b) 'consumer' means any natural person who, in contracts covered by this Directive, is acting for purposes which are outside his trade, business or profession;

(c) 'seller or supplier' means any natural or legal person who, in contracts covered by this Directive, is acting for purposes relating to his trade, business or profession, whether publicly owned or privately owned.

Article 3

1. A contractual term which has not been individually negotiated shall be regarded as unfair if, contrary to the requirement of good faith, it causes a significant imbalance in the parties' rights and obligations arising under the contract, to the detriment of the consumer.

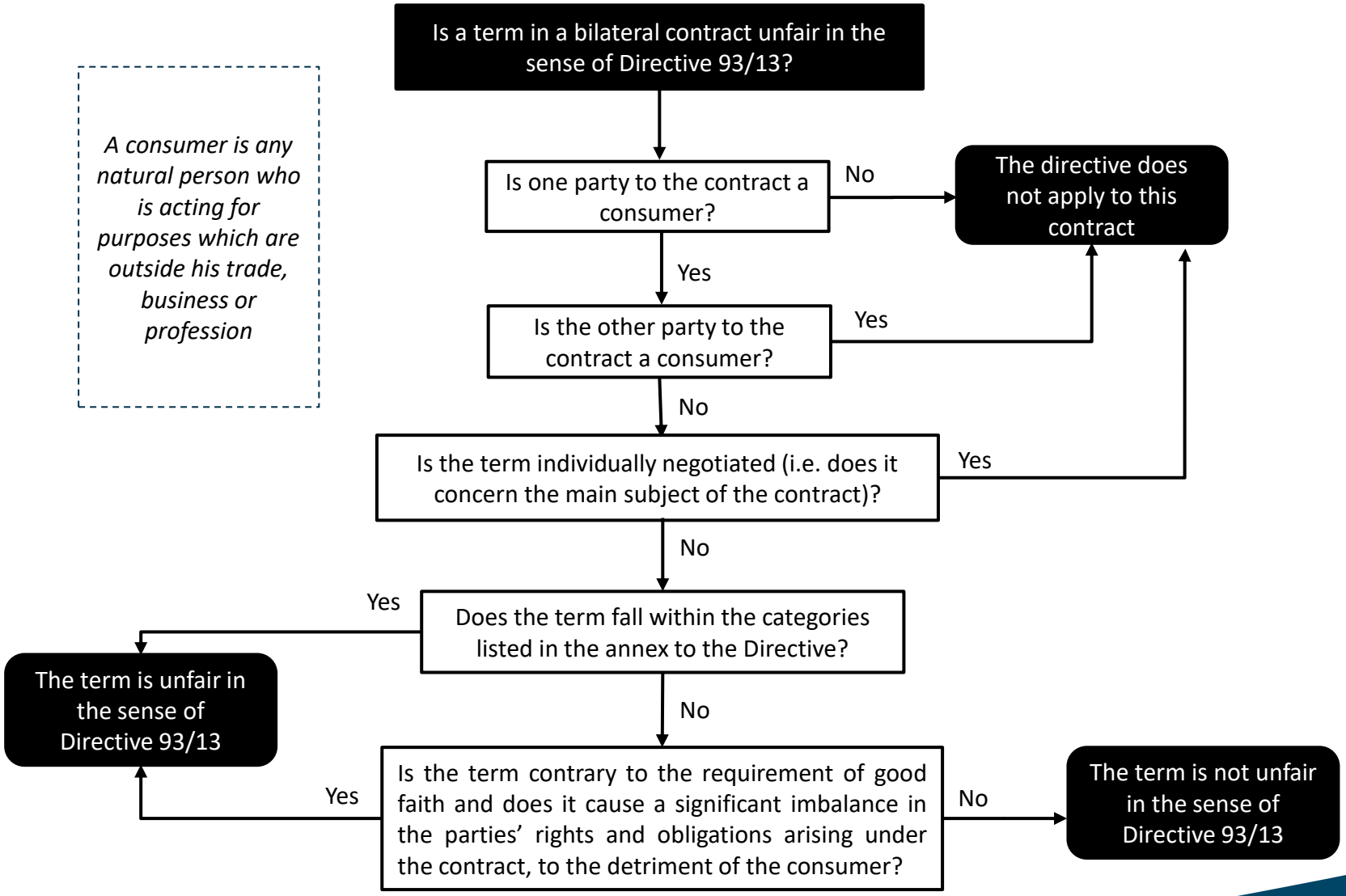
2. A term shall always be regarded as not individually negotiated where it has been drafted in advance and the consumer has therefore not been able to influence the substance of the term, particularly in the context of a pre-formulated standard contract.

The fact that certain aspects of a term or one specific term have been individually negotiated shall not exclude the application of this Article to the rest of a contract if an overall assessment of the contract indicates that it is nevertheless a pre-formulated standard contract.

Where any seller or supplier claims that a standard term has been individually negotiated, the burden of proof in this respect shall be incumbent on him.

3. The Annex shall contain an indicative and non-exhaustive list of the terms which may be regarded as unfair.

Q8 - Unfair contract terms



Summary Part 3

Coding decision trees:

- First problem analysis
- Then coding the logic

Current developments:

- Using generative AI to write code
  - Need to know how to phrase (divide in blocks)
  - Understand code and make small modifications

Summary

Python has many powerful libraries:

- Easy to use : *import [name of library]*

Learn more?

- Online courses; e.g. : https://snakify.org/
- You can use genAI to assist your code writing
  - Debug your code, write tests
  - Use it for boring, repetitive tasks
  - Develop an interface for your program
  - Suggest libraries to use

You

Write a Python program that checks if a given year is a leap year or not. A leap year is divisible by 4, but not by 100, unless it is divisible by 400.

ChatGPT

Here's the Python program that checks if a given year is a leap year:

python Copy code

```python
def is_leap_year(year):
    """
    Check if a given year is a leap year.

    Parameters:
    year (int): The year to check.

    Returns:
    bool: True if year is a leap year, False otherwise.
    """
    # If a year is divisible by 400, it's a leap year
    if year % 400 == 0:
        return True
    # If a year is divisible by 100 but not by 400, it's not a leap year
    if year % 100 == 0:
        return False
    # If a year is divisible by 4 but not by 100, it's a leap year
    if year % 4 == 0:
        return True
    # All other years are not leap years
    return False

# For demonstration, let's check the leap year status of a range of years
```

University of Antwerp